

Data structures and Algorithms Rapid-Fire Interview Questions & Answers

Comprehensive Rapid-Fire Bank

Format: Interviewer asks → Candidate answers in 1–3 sentences.

Goal: Fast revision, concept recognition, complexity recall, and coding-interview pattern identification.

created with help from ai in hope that someone will benefit

1. Fundamentals

Q1. What is an algorithm?

A: A finite sequence of well-defined steps that transforms input into the required output.

Q2. What is a data structure?

A: A way of organizing and storing data so operations such as access, insertion, deletion, and search can be performed efficiently.

Q3. What is time complexity?

A: It describes how an algorithm's running time grows as the input size grows.

Q4. What is space complexity?

A: It measures the additional memory an algorithm requires as a function of input size.

Q5. What does Big-O represent?

A: Big-O gives an asymptotic upper bound on growth, commonly used to describe worst-case complexity.

Q6. What does Big-Theta represent?

A: Theta gives a tight asymptotic bound when both upper and lower bounds have the same growth rate.

Q7. What does Big-Omega represent?

A: Omega gives an asymptotic lower bound on growth.

Q8. What is constant time?

A: $O(1)$; the operation takes approximately the same amount of work regardless of input size.

Q9. What is linear time?

A: $O(n)$; work grows proportionally with the input size.

Q10. What is logarithmic time?

A: $O(\log n)$; each step typically reduces the remaining search space by a constant factor, as in binary search.

Q11. What is quadratic complexity?

A: $O(n^2)$, commonly produced by two nested loops over the same input.

Q12. What is amortized complexity?

A: It is the average cost per operation over a sequence of operations, even when some individual operations are expensive.

Q13. What is recursion?

A: A technique where a function solves a problem by calling itself on a smaller instance of the problem.

Q14. What is a base case?

A: The condition that stops recursive calls and directly returns a result.

Q15. What happens if recursion has no terminating condition?

A: It can recurse indefinitely until stack overflow or another runtime failure occurs.

Q16. What is a recurrence relation?

A: An equation that expresses the cost or result of a recursive problem in terms of smaller instances.

Q17. Why can recursion use extra memory?

A: Each active call consumes a stack frame, so recursion depth contributes to auxiliary space.

Q18. What is an object?

A: An object is an instance of a class containing state and behavior.

Q19. What is a class?

A: A class is a blueprint defining the data and operations associated with objects.

Q20. Why analyze asymptotic complexity?

A: It lets us compare algorithms independently of machine speed and implementation-specific constant factors.

2. Mathematics, Patterns & Recursion

Q21. How do you check whether an integer is even?

A: Check whether $n \% 2 == 0$.

Q22. What is the $O(1)$ formula for sum of first n natural numbers?

A: $n(n+1)/2$.

Q23. How can you find a number closest to n that is divisible by m ?

A: Consider the nearest multiples around n , typically using quotient/remainder arithmetic.

Q24. How can an integer be represented as a sum of consecutive positive integers?

A: Test possible sequence lengths or use divisor properties; the problem can be reduced to finding suitable factorization of the number.

Q25. What is the basic approach for a solid rectangle pattern?

A: Use nested loops: outer loop for rows and inner loop for columns.

Q26. What is Floyd's triangle?

A: A triangular pattern containing consecutive integers, with row i containing i values.

Q27. How do you print a hollow rectangle?

A: Print a symbol on the border and spaces inside; first/last row and first/last column are the boundary.

Q28. How do you print 1 to n recursively?

A: Recurse toward the base case first and print while unwinding, or print before recursion depending on the required order.

Q29. How do you print n to 1 recursively?

A: Print n and recursively process $n-1$ until reaching the base case.

Q30. What is factorial recurrence?

A: $\text{fact}(n) = n * \text{fact}(n-1)$, with $\text{fact}(0)=1$.

Q31. What is Euclid's algorithm for GCD?

A: Repeatedly replace (a, b) with $(b, a \% b)$ until b becomes zero.

Q32. What is the complexity of Euclid's GCD algorithm?

A: $O(\log \min(a, b))$.

Q33. How do you calculate x^n efficiently?

A: Use exponentiation by squaring, reducing the exponent roughly by half at each step.

Q34. What is the complexity of exponentiation by squaring?

A: $O(\log n)$.

Q35. How do you count digits of a positive integer?

A: Repeatedly divide by 10 and count divisions, or use logarithms where appropriate.

Q36. What is a prime number?

A: A number greater than 1 having exactly two positive divisors: 1 and itself.

Q37. How do you test primality efficiently?

A: Check divisibility only up to $\sqrt{n}$ after handling small cases.

Q38. What is an Armstrong number?

A: A number equal to the sum of its digits raised to the number of digits.

Q39. How are trailing zeros of $n!$ determined?

A: Count factors of 5: $\text{floor}(n/5) + \text{floor}(n/25) + \text{floor}(n/125) + \dots$

Q40. Why count factors of 5 rather than 10 for $n!$?

A: Factors of 2 are more abundant than factors of 5, so each factor of 5 can pair with a 2 to create a trailing zero.

Q41. How do you generate prime factors?

A: Trial-divide by increasing possible factors, usually up to $\sqrt{n}$, while repeatedly removing each factor.

Q42. How do you enumerate all divisors efficiently?

A: Iterate i from 1 through $\sqrt{n}$; if i divides n , add i and n/i .

Q43. What is a palindrome number?

A: A number that reads identically from left to right and right to left.

Q44. What is Tower of Hanoi?

A: A recursive puzzle requiring moving n disks between pegs while never placing a larger disk on a smaller one.

Q45. What is the minimum number of moves for Tower of Hanoi?

A: $2^n - 1$.

Q46. What is the Josephus problem?

A: People arranged in a circle are eliminated at a fixed interval; the task is to find the survivor.

Q47. What is the recurrence for Josephus using zero-based indexing?

A: $J(n, k) = (J(n-1, k) + k) \% n$, with $J(1, k) = 0$.

Q48. What is the key idea of the water-jug problem?

A: Model filling, emptying, and pouring as states and transitions, or use the GCD property when appropriate.

Q49. How are non-attacking knights counted?

A: Determine placements where no pair of knights attacks each other; on standard boards, attacking relationships depend on relative row/column offsets of 1 and 2.

Q50. What is a butterfly pattern?

A: A symmetric pattern combining increasing/decreasing star blocks and spaces.

3. Arrays, Strings & Matrices

Q51. What is an array?

A: A contiguous collection of elements of the same logical type, providing $O(1)$ indexed access.

Q52. What is the main advantage of arrays?

A: Fast random access through indexing.

Q53. What is the main disadvantage of arrays?

A: Inserting or deleting elements in the middle can require shifting many elements.

Q54. What is a string?

A: A sequence of characters.

Q55. How do you check if an array is sorted?

A: Scan adjacent pairs and ensure each pair satisfies the required ordering.

Q56. How do you reverse an array in place?

A: Use two pointers from both ends and swap until they meet.

Q57. How do you reverse an array in groups of k?

A: Reverse each block of k elements, with the handling of the final partial block determined by the problem specification.

Q58. How can an array be rotated efficiently?

A: The reversal algorithm rotates in $O(n)$ time and $O(1)$ extra space.

Q59. How do you generate all subarrays?

A: Choose every start/end pair; there are $O(n^2)$ subarrays.

Q60. How many subarrays does an array of n elements have?

A: $n(n+1)/2$.

Q61. How do you arrange positive and negative numbers by sign?

A: Use partitioning/two-pointer techniques; stable and in-place variants have different trade-offs.

Q62. What is the best solution for stock profit with one transaction?

A: Track the minimum price seen so far and maximize `currentPrice - minPrice`.

Q63. What is the complexity of one-transaction stock profit?

A: $O(n)$ time and $O(1)$ extra space.

Q64. How do you maximize profit with unlimited transactions?

A: Add every positive day-to-day increase, or use equivalent valley-peak logic.

Q65. What is a leader in an array?

A: An element that is greater than all elements to its right, according to the common definition.

Q66. How do you find leaders efficiently?

A: Scan from right to left while maintaining the maximum seen.

Q67. How do you compare two strings?

A: Compare lengths/content according to the required equality semantics; for character-for-character equality, every corresponding character must match.

Q68. How do you check whether a string is a palindrome?

A: Compare characters using two pointers moving inward.

Q69. How do you toggle character case?

A: Convert lowercase to uppercase and uppercase to lowercase using character operations or library functions.

Q70. How do you remove all occurrences of a character?

A: Scan the string and copy only characters that are not equal to the target.

Q71. How do you remove spaces from a string?

A: Build a result containing all non-space characters.

Q72. What is a subsequence?

A: A sequence obtained by deleting zero or more characters/elements without changing the relative order.

Q73. How do you test whether s is a subsequence of t?

A: Scan t with a pointer into s and advance the pointer whenever characters match.

Q74. What is the first non-repeating character problem?

A: Find the first character whose frequency is exactly one.

Q75. How do you solve first non-repeating character efficiently?

A: Count frequencies first, then scan the string again.

Q76. What is a pangram?

A: A string containing every letter of the target alphabet at least once.

Q77. What is a majority element?

A: An element appearing more than $n/2$ times.

Q78. Which algorithm finds a majority element in $O(n)$ time and $O(1)$ space?

A: Boyer–Moore voting algorithm, followed by verification if required.

Q79. What is Majority Element II?

A: Find all elements occurring more than $n/3$ times.

Q80. How many candidates can occur more than $n/3$ times?

A: At most two.

Q81. What is Kadane's algorithm?

A: A linear-time algorithm for maximum subarray sum that maintains the best subarray ending at the current position.

Q82. What is the recurrence idea behind Kadane?

A: Either extend the previous subarray or start a new subarray at the current element.

Q83. How do you sum all subarray sums efficiently?

A: Each element contributes $a[i] * (i+1) * (n-i)$ to the total.

Q84. Why does the contribution formula work?

A: There are $(i+1)$ possible starting positions and $(n-i)$ possible ending positions containing index i .

Q85. Why is maximum product subarray harder than maximum sum?

A: Negative values can turn a minimum product into the maximum after multiplication by another negative.

Q86. What should maximum-product tracking maintain?

A: Both the maximum and minimum product ending at the current position.

Q87. What is `atoi`?

A: Conversion of a numeric string representation into an integer, including handling of signs and invalid/overflow cases according to the specification.

Q88. What is `URLify`?

A: Replacing spaces in a string with an encoded representation such as `%20`, usually under a given length constraint.

Q89. How do you multiply very large integers represented as strings?

A: Perform grade-school digit multiplication using an integer result array, avoiding built-in integer overflow.

Q90. What is matrix spiral traversal?

A: Traverse the matrix layer by layer using top, bottom, left, and right boundaries.

Q91. What is matrix transpose?

A: Convert $A[i][j]$ into $A[j][i]$.

Q92. How do you rotate a square matrix 90° clockwise in place?

A: Transpose the matrix and then reverse every row.

Q93. What is a Toeplitz matrix?

A: A matrix in which every diagonal from top-left to bottom-right contains equal values.

Q94. What is boundary traversal?

A: Visit the outer boundary of the matrix without duplicating corner elements.

Q95. What is Conway's Game of Life?

A: A cellular-automaton simulation where each cell's next state depends on the number of live neighboring cells.

Q96. What is the Set Matrix Zero problem?

A: If a cell is zero, set its entire row and column to zero, preferably using $O(1)$ auxiliary space.

Q97. What is a circular subarray?

A: A subarray that may wrap from the end of the array back to the beginning.

Q98. How is maximum circular subarray sum commonly solved?

A: $\max(\text{normalKadane}, \text{totalSum} - \text{minimumSubarraySum})$, with a special case when all elements are negative.

Q99. What is lexicographic rank?

A: The position of a string among permutations ordered lexicographically, usually counting smaller permutations before it.

Q100. What is text justification?

A: Arrange words into fixed-width lines with spaces distributed according to the specified justification rules.

4. Searching

Q101. What is linear search?

A: Sequentially inspect elements until the target is found or the array ends.

Q102. Complexity of linear search?

A: $O(n)$ worst case and $O(1)$ extra space.

Q103. How do you find the largest element?

A: Maintain a running maximum in one scan.

Q104. How do you find the second largest distinct element?

A: Maintain the largest and second-largest values while handling duplicates carefully.

Q105. What is a local maximum?

A: An element that satisfies the required greater-than relationship with its neighbors.

Q106. What is binary search?

A: A search algorithm on ordered data that repeatedly halves the candidate search range.

Q107. Complexity of binary search?

A: $O(\log n)$ time and $O(1)$ iterative auxiliary space.

Q108. What is insertion position?

A: The first index where a target can be inserted without violating sorted order.

Q109. What is lower bound?

A: The first position containing a value greater than or equal to the target.

Q110. What is upper bound?

A: The first position containing a value strictly greater than the target.

Q111. How do you count occurrences in a sorted array?

A: `upper_bound(x) - lower_bound(x)`.

Q112. How do you find a single element in a sorted array where others appear twice?

A: Use XOR or exploit pair-index structure with binary search.

Q113. How can XOR find the missing number from 0..n?

A: XOR all indices and values; paired values cancel, leaving the missing number.

Q114. How do you search a rotated sorted array?

A: At each step determine which half is sorted, then decide which half can contain the target.

Q115. Why is rotated-array search with duplicates harder?

A: Equal boundary values can make it impossible to determine which half is sorted, requiring boundary contraction.

Q116. What is a peak element?

A: An element that is not smaller than its relevant neighbors.

Q117. Can binary search find a peak?

A: Yes; compare adjacent elements and move toward the rising side.

Q118. How do you search a row-column sorted matrix?

A: Start at the top-right or bottom-left and eliminate a row or column at every step.

Q119. What is search on answer?

A: Binary search over a numeric answer space when feasibility is monotonic.

Q120. What is the key question before using binary search on answer?

A: Whether a candidate answer can be checked efficiently and whether feasibility changes monotonically.

Q121. How do you compute integer square root with binary search?

A: Search the largest integer x such that $x^2 \leq n$, using overflow-safe multiplication.

Q122. What is the common pattern in Koko Eating Bananas?

A: Binary search the eating speed and test whether all bananas can be eaten within the allowed time.

Q123. What is the Book Allocation pattern?

A: Binary search the maximum pages assigned to one student and greedily check whether allocation is feasible.

Q124. What is Aggressive Cows?

A: Maximize the minimum distance between selected positions using binary search on distance.

Q125. What is median of two sorted arrays?

A: Partition the two arrays so that the left halves contain half the total elements and all left values are $\leq$ all right values.

Q126. Complexity of optimal median-of-two-sorted-arrays solution?

A: $O(\log \min(m,n))$ time and $O(1)$ extra space.

Q127. What is the kth element of two sorted arrays?

A: Find a partition where exactly k elements lie on the left and partition boundary ordering is valid.

Q128. What is the Kth missing element problem?

A: Use the count of missing values before an index and binary search for the appropriate position.

Q129. Why is monotonicity important in search-on-answer?

A: It allows the candidate space to be divided into feasible and infeasible regions.

Q130. What is a common overflow issue in binary search?

A: Computing $(low+high)/2$ can overflow; use $low + (high-low)/2$.

5. Sorting

Q131. What is sorting?

A: Arranging elements according to an ordering relation.

Q132. What is Quick Sort?

A: A divide-and-conquer sorting algorithm that partitions around a pivot and recursively sorts the partitions.

Q133. Average complexity of Quick Sort?

A: $O(n \log n)$.

Q134. Worst-case complexity of Quick Sort?

A: $O(n^2)$, commonly caused by consistently poor pivot choices.

Q135. How can Quick Sort's worst case be reduced?

A: Use randomized or better pivot selection and balanced partitioning.

Q136. What is Merge Sort?

A: Divide the array into halves, recursively sort them, then merge the sorted halves.

Q137. Complexity of Merge Sort?

A: $O(n \log n)$ time.

Q138. Is Merge Sort stable?

A: Yes, when the merge operation preserves the relative order of equal elements.

Q139. What extra space does array Merge Sort usually require?

A: $O(n)$ auxiliary space.

Q140. What is the Dutch National Flag problem?

A: Sort an array containing 0, 1, and 2 in one pass using three regions.

Q141. Complexity of Dutch National Flag sorting?

A: $O(n)$ time and $O(1)$ extra space.

Q142. How do you move zeros to the end?

A: Maintain a write position for nonzero elements and fill the remaining positions with zeros.

Q143. What is cycle sort known for?

A: Minimizing the number of writes.

Q144. What is the minimum missing positive problem?

A: Find the smallest positive integer absent from an unsorted array, commonly in $O(n)$ time and $O(1)$ extra space using index placement.

Q145. What is inversion count?

A: The number of pairs (i, j) such that $i < j$ and $a[i] > a[j]$.

Q146. How can inversion count be computed efficiently?

A: Modify Merge Sort and count cross-half inversions during merging.

Q147. Complexity of Merge-Sort inversion counting?

A: $O(n \log n)$.

Q148. What is interval merging?

A: Sort intervals by start and merge overlapping intervals into maximal non-overlapping ranges.

Q149. What is the complexity of interval merging?

A: $O(n \log n)$ due to sorting, followed by $O(n)$ merging.

Q150. How do you insert an interval into sorted non-overlapping intervals?

A: Add intervals before it, merge all overlapping intervals, then add remaining intervals.

Q151. What is the largest-number-from-array problem?

A: Sort numbers using comparator $a+b > b+a$.

Q152. Why does the largest-number comparator use concatenation?

A: Comparing $a+b$ and $b+a$ directly determines which ordering produces the larger combined number.

Q153. What is the meeting-room problem?

A: Determine whether intervals overlap; sorting start/end events or intervals is a common approach.

Q154. What is minimum platforms?

A: Find the maximum number of overlapping train arrival/departure intervals.

Q155. What is the rotation maximum $\text{sum}(i \cdot \text{arr}[i])$ pattern?

A: Derive a recurrence between consecutive rotations to achieve $O(n)$.

6. Bit Manipulation

Q156. What are bitwise operators?

A: Operators that manipulate individual binary bits, such as AND, OR, XOR, NOT, left shift, and right shift.

Q157. What does $x \& 1$ tell you?

A: It tells whether the least significant bit is set, which identifies odd/even for nonnegative integers.

Q158. What does XOR do with equal values?

A: $x \oplus x = 0$.

Q159. What is $x \oplus 0$?

A: x .

Q160. Why is XOR useful for finding an odd-occurring element?

A: Equal elements cancel in XOR, leaving the value whose occurrence count is odd.

Q161. How do you check whether the k th bit is set?

A: Test $(n \& (1 \ll k)) \neq 0$, assuming zero-based bit indexing.

Q162. How do you isolate the lowest set bit?

A: $n \& -n$ for standard two's-complement integer representation.

Q163. How do you check whether n is a power of two?

A: For positive n , $n \& (n-1) == 0$.

Q164. Why does the power-of-two test work?

A: A power of two has exactly one set bit; subtracting one clears it and sets lower bits.

Q165. What is the only-set-bit position problem?

A: Determine the position when a number contains exactly one set bit.

Q166. How do you count set bits efficiently?

A: Brian Kernighan's algorithm repeatedly clears the lowest set bit.

Q167. Complexity of Brian Kernighan's algorithm?

A: $O(\text{number of set bits})$.

Q168. What is bit rotation?

A: Bits shifted out from one end are reintroduced at the opposite end.

Q169. What is a binary palindrome?

A: A number whose binary representation reads identically in both directions.

Q170. What is Hamming distance?

A: The number of bit positions at which two binary representations differ.

Q171. How is Hamming distance computed?

A: Count set bits in $x \oplus y$.

Q172. What is Gray code?

A: A sequence where consecutive values differ in exactly one bit.

Q173. Formula for binary-reflected Gray code?

A: $\text{gray}(i) = i \oplus (i \gg 1)$.

Q174. How can two odd-occurring numbers be found?

A: XOR all values to obtain $x \oplus y$, isolate a set bit, and partition values by that bit.

Q175. What is the key idea for two unique numbers among duplicates?

A: XOR isolates the XOR of the two unique values, then a distinguishing bit separates them.

Q176. How can a missing and repeating number be found using bit techniques?

A: XOR-based separation can distinguish the two exceptional values, followed by identifying which is missing versus repeated.

Q177. What is subset XOR behavior?

A: XOR is associative and commutative, enabling useful cancellation and contribution-counting techniques.

Q178. What is Sudoku validation with bit manipulation?

A: Represent used digits in rows, columns, and boxes as bit masks.

7. Hashing

Q179. What is hashing?

A: Mapping keys to positions/buckets using a hash function for fast average-case lookup.

Q180. What is a collision?

A: Two distinct keys map to the same hash bucket.

Q181. What is separate chaining?

A: Each bucket stores multiple colliding entries, commonly using a linked structure or dynamic container.

Q182. What is linear probing?

A: On collision, inspect subsequent slots until an appropriate slot is found.

Q183. What is quadratic probing?

A: Probe positions using quadratic offsets to reduce clustering compared with simple linear probing.

Q184. What is load factor?

A: The ratio of stored entries to available hash-table buckets.

Q185. Why does a hash table resize?

A: To keep the load factor controlled and preserve efficient average lookup performance.

Q186. How do you find distinct elements?

A: Insert elements into a set or track them using a hash table.

Q187. How do you test whether one array is a subset of another?

A: Track frequencies of the larger collection and consume matching frequencies from the smaller collection.

Q188. How do you compute array union using hashing?

A: Insert elements from both arrays into a set.

Q189. How do you compute intersection with frequencies?

A: Count one array and consume counts while scanning the other.

Q190. What is the Two Sum hashing approach?

A: Store previously seen values and look for `target-current` during one pass.

Q191. Complexity of Two Sum using hashing?

A: $O(n)$ expected time and $O(n)$ space.

Q192. What is Fizz Buzz?

A: Output a rule-based sequence, commonly replacing multiples of 3 and 5 with words.

Q193. How do you pair values whose sums are divisible by k ?

A: Group numbers by remainder modulo k and match complementary remainders.

Q194. What special case exists for remainder 0?

A: All remainder-0 values pair among themselves.

Q195. What is an anagram?

A: Two strings are anagrams if they contain the same characters with the same frequencies.

Q196. How can anagrams be checked efficiently?

A: Use a frequency array/map, or sort both strings.

Q197. What are isomorphic strings?

A: Characters in one string can be mapped one-to-one to characters in the other while preserving positions.

Q198. Why may two maps be needed for isomorphic strings?

A: To ensure the mapping is one-to-one in both directions.

Q199. How do you find the most frequent element?

A: Count frequencies and track the maximum.

Q200. What is the longest consecutive subsequence pattern?

A: Put values in a hash set and start sequences only from values whose predecessor is absent.

8. Two-Pointer Technique

Q201. What is the two-pointer technique?

A: Maintain two indices that move according to a condition to avoid unnecessary nested loops.

Q202. Where is two-pointer most effective?

A: Sorted arrays, strings, linked lists, partitioning, and problems involving a moving boundary.

Q203. How do you solve Two Sum in a sorted array?

A: Start at both ends; move left/right according to whether the current sum is smaller/larger than target.

Q204. Complexity of sorted Two Sum?

A: $O(n)$ time and $O(1)$ extra space.

Q205. How do you find a minimum-length subarray with sum greater than x?

A: For positive values, use a variable sliding window with two pointers.

Q206. How do you remove duplicates from a sorted array in place?

A: Maintain a slow write pointer and a fast scan pointer.

Q207. How do you check a sentence palindrome?

A: Use two pointers while skipping spaces, punctuation, and applying the required case normalization.

Q208. What is 3 Sum?

A: Find triplets satisfying a target sum, commonly by sorting and using a fixed element plus two pointers.

Q209. Complexity of standard 3 Sum?

A: $O(n^2)$ after sorting.

Q210. How is 4 Sum commonly solved?

A: Sort, fix two values, then use two pointers for the remaining pair.

Q211. Complexity of standard 4 Sum?

A: $O(n^3)$.

Q212. What is the container-with-most-water insight?

A: Area is limited by the shorter boundary; move the pointer at the shorter boundary.

Q213. Why move the shorter boundary?

A: Moving the taller boundary cannot increase the limiting height while width decreases.

Q214. How do you find common elements in three sorted arrays?

A: Maintain three pointers and advance the pointer holding the smallest value.

Q215. What is a closest pair from two sorted arrays?

A: Use two pointers from opposite ends and shrink the difference based on the current sum.

Q216. What is the celebrity problem?

A: Find a person known by everyone but who knows nobody.

Q217. How can celebrity be solved in $O(n)$?

A: Eliminate candidates pairwise, then verify the remaining candidate.

Q218. What is the policemen-and-thieves pattern?

A: Use pointers to match eligible policemen and thieves within the allowed distance.

Q219. What is closest 3-sum?

A: Sort the array, fix one element, and use two pointers while tracking the closest sum.

Q220. Why sort before many two-pointer problems?

A: Ordering provides directional information that lets pointers eliminate large portions of the search space.

9. Sliding Window

Q221. What is a sliding window?

A: A moving contiguous range used to maintain information about a subarray/substring efficiently.

Q222. What is a fixed-size window?

A: A window whose length remains exactly k .

Q223. How do you find maximum sum of every k -sized window?

A: Add the entering element and subtract the leaving element as the window moves.

Q224. Complexity of fixed-size window maximum sum?

A: $O(n)$ time and $O(1)$ extra space.

Q225. How do you count distinct values in each window?

A: Maintain a frequency map while adding the right element and removing the left element.

Q226. What is a variable-size window?

A: A window whose left boundary moves conditionally while the right boundary expands.

Q227. When does the standard sum-based sliding window work especially well?

A: When values are nonnegative and the condition is monotonic as the window expands.

Q228. What is the "maximize consecutive ones with k flips" pattern?

A: Expand the window while zero count $\leq k$; shrink from the left when it exceeds k .

Q229. What is Fruit Into Baskets?

A: Find the longest contiguous range containing at most two distinct values.

Q230. What is the general longest-window pattern?

A: Expand right, maintain validity, and shrink left until the window becomes valid again.

Q231. What is the minimum window substring problem?

A: Find the smallest substring containing all required characters with required frequencies.

Q232. How is Minimum Window Substring typically solved?

A: Use frequency counts and two pointers, tracking how many required character counts are satisfied.

Q233. What is Minimum Window Subsequence?

A: Find the shortest substring of one string containing another string as a subsequence.

Q234. What is "at most K distinct" useful for?

A: Exact-K distinct substring counts can often be computed as $\text{atMost}(K) - \text{atMost}(K-1)$.

Q235. How do you find the longest substring without repeating characters?

A: Maintain last-seen positions or frequencies and move the left pointer past duplicates.

Q236. What is character replacement sliding window?

A: Maintain the highest character frequency and ensure $\text{windowLength} - \text{maxFrequency} \leq k$.

Q237. What is the key idea of counting subarrays with product $< k$?

A: For positive numbers, expand the window and shrink while the product is $\geq k$; every valid ending contributes window length.

Q238. How do you count subarrays with exactly k odd numbers?

A: Use prefix counts or an at-most-k transformation.

Q239. What is the first negative integer in every window?

A: Maintain a queue/deque of indices of negative values inside the current window.

Q240. What is the MEX?

A: The minimum excluded nonnegative integer.

Q241. What is the chocolate distribution pattern?

A: Sort packets and find the minimum difference between maximum and minimum in every window of size m .

Q242. Why does sorting solve chocolate distribution?

A: In sorted order, the minimum range for m chosen elements is represented by a contiguous window.

Q243. What is a deque's role in sliding-window maximum?

A: Maintain candidate indices in decreasing value order so the front is always the maximum.

Q244. Complexity of sliding-window maximum with a monotonic deque?

A: $O(n)$.

Q245. Why is each element pushed/popped at most once in a monotonic deque?

A: Once an element is removed from the deque it can never become relevant again for a later window.

10. Prefix Sum, Difference Array & Hashing

Q246. What is a prefix sum?

A: A cumulative sum where each position stores the sum of all elements up to that position.

Q247. Why use prefix sums?

A: They turn repeated range-sum queries from $O(n)$ each into $O(1)$ after $O(n)$ preprocessing.

Q248. How is a range sum $[l,r]$ obtained?

A: With a prefix array: `prefix[r] - prefix[l-1]`, using an appropriate `prefix[-1]` convention.

Q249. What is a 2D prefix sum?

A: A cumulative sum over rows and columns allowing rectangular region sums in $O(1)$.

Q250. What is an equilibrium index?

A: An index where the sum of elements to the left equals the sum to the right.

Q251. How do you find an equilibrium index in $O(n)$?

A: Maintain total sum and a running left sum.

Q252. What is product of array except self?

A: For every index, compute the product of all elements except the current one, commonly using prefix and suffix products.

Q253. How can product-except-self avoid division?

A: First store prefix products, then multiply each result by a running suffix product.

Q254. What is a difference array?

A: An auxiliary array representing changes between adjacent positions, enabling $O(1)$ range updates.

Q255. How is a range increment $[l,r]$ represented in a difference array?

A: Add the increment at l and subtract it after r .

Q256. How are final values recovered from a difference array?

A: Take its prefix sum.

Q257. What is a 2D difference array?

A: A two-dimensional extension that supports rectangle updates efficiently using corner updates.

Q258. What is a zero-sum subarray?

A: A contiguous subarray whose elements sum to zero.

Q259. How does prefix-sum hashing detect a zero-sum subarray?

A: If the same prefix sum appears twice, the elements between those positions sum to zero.

Q260. How do you find the longest zero-sum subarray?

A: Store the first index where each prefix sum occurs and maximize repeated-prefix distance.

Q261. How do you find the longest subarray with equal 0s and 1s?

A: Treat 0 as -1 and 1 as $+1$, then find the longest repeated prefix sum.

Q262. How do you find the longest subarray with sum divisible by k?

A: Track the earliest index for each prefix-sum remainder modulo k.

Q263. Why normalize negative modulo values?

A: Programming-language remainder can be negative; normalize to a consistent range such as $0..k-1$.

Q264. How do you find the longest subarray with sum K?

A: Store first occurrence of prefix sums and look for $\text{prefix} - K$.

Q265. How do you count subarrays with sum K?

A: Maintain a frequency map of prefix sums and add the count of $\text{prefix} - K$.

Q266. How do you count subarrays with a target XOR?

A: Maintain prefix XOR frequencies and look for $\text{prefixXor} \oplus \text{target}$.

Q267. Why does prefix XOR work like prefix sum?

A: XOR is associative and self-inverting: $a \oplus a = 0$.

Q268. What is the largest zero-sum submatrix idea?

A: Compress rows/columns and reduce the 2D problem to repeated 1D zero-sum subarray detection.

Q269. What is maximum-sum rectangle?

A: Compress a pair of rows/columns and apply Kadane's algorithm to the resulting 1D array.

Q270. What is the key trade-off of prefix preprocessing?

A: Extra memory/time preprocessing is exchanged for much faster repeated queries.

11. Backtracking

Q271. What is backtracking?

A: Explore a candidate solution, and undo the choice when it cannot lead to a valid solution.

Q272. What is the difference between recursion and backtracking?

A: Recursion is a control technique; backtracking is a systematic search strategy that uses recursion or an equivalent mechanism with choices and undo operations.

Q273. What is the basic backtracking template?

A: Choose → recurse → undo.

Q274. What is a permutation?

A: An arrangement where order matters and each selected item occupies a distinct position.

Q275. Complexity of generating all permutations of n distinct elements?

A: $O(n \cdot n!)$ if output construction/copying is counted.

Q276. What is the Rat in a Maze problem?

A: Find paths through allowed cells from source to destination while avoiding blocked cells.

Q277. What is Word Search?

A: Determine whether a word can be formed by adjacent grid cells without reusing a cell.

Q278. Why mark a cell visited in Word Search?

A: To prevent using the same grid cell more than once in the current path.

Q279. What is N-Queens?

A: Place N queens on an $N \times N$ board so that no two queens attack each other.

Q280. What must be checked for N-Queens?

A: Column, main diagonal, and anti-diagonal conflicts.

Q281. How can N-Queens be optimized?

A: Use boolean sets/bit masks for columns and diagonals instead of scanning the board.

Q282. What is Sudoku solving?

A: Assign digits to empty cells while maintaining row, column, and box constraints.

Q283. What is M-Coloring?

A: Determine whether graph vertices can be colored with at most M colors so adjacent vertices differ.

Q284. What is Knight's Tour?

A: Visit every chessboard square exactly once using legal knight moves.

Q285. What is palindromic partitioning?

A: Split a string into substrings where every substring is a palindrome.

Q286. What is pruning?

A: Eliminating a search branch as soon as it is known not to produce a valid/better solution.

Q287. Why is pruning important?

A: It can dramatically reduce the exponential search space.

Q288. What is Word Break conceptually?

A: Determine whether a string can be segmented into dictionary words; it can be solved using DP or backtracking with memoization.

Q289. What is memoized backtracking?

A: Cache results for repeated states so identical subproblems are not solved repeatedly.

Q290. What is the difference between combinations and permutations?

A: Combinations ignore order; permutations consider order.

12. Linked Lists

Q291. What is a linked list?

A: A sequence of nodes where each node stores data and one or more links to other nodes.

Q292. Main advantage over arrays?

A: Efficient insertion/deletion when the relevant node/position is already known, without shifting subsequent elements.

Q293. Main disadvantage?

A: No $O(1)$ random access and additional pointer memory.

Q294. What is a singly linked list?

A: Each node has data and a pointer/reference to the next node.

Q295. What is a doubly linked list?

A: Each node has links to both previous and next nodes.

Q296. What is a circular linked list?

A: The last node points back to the first node.

Q297. How do you reverse a singly linked list iteratively?

A: Maintain previous, current, and next pointers and reverse each link.

Q298. Complexity of linked-list reversal?

A: $O(n)$ time and $O(1)$ extra space.

Q299. How do you find the middle node?

A: Use slow and fast pointers; slow moves one step while fast moves two.

Q300. How do you remove the nth node from the end?

A: Maintain two pointers separated by n positions, then remove when the fast pointer reaches the end.

Q301. How do you detect a linked-list cycle?

A: Floyd's tortoise-and-hare algorithm.

Q302. Complexity of Floyd cycle detection?

A: $O(n)$ time and $O(1)$ space.

Q303. How do you find cycle length?

A: After detecting a meeting point, walk one pointer around the cycle until it returns to the meeting point.

Q304. How do you check whether a linked list is a palindrome?

A: Find the middle, reverse the second half, and compare both halves.

Q305. How do you find intersection of two linked lists?

A: Use two pointers that switch heads after reaching the end, equalizing total traversal distance.

Q306. What is K-group reversal?

A: Reverse linked-list nodes in groups of k while preserving the required ordering between groups.

Q307. How do you merge two sorted linked lists?

A: Repeatedly attach the smaller current node.

Q308. How do you merge k sorted linked lists efficiently?

A: Use a min-heap or divide-and-conquer merging.

Q309. Complexity of heap-based merge k lists?

A: $O(N \log k)$, where N is the total number of nodes.

Q310. How do you add two numbers represented by linked lists?

A: Traverse both lists, add corresponding digits plus carry, and build the result.

Q311. What is linked-list rotation?

A: Move the last k nodes to the front after reducing k modulo list length.

Q312. How do you clone a list with random pointers?

A: Use a mapping from old nodes to new nodes, or interleave cloned nodes with originals for $O(1)$ auxiliary space.

Q313. What is LRU Cache?

A: A cache that evicts the least recently used item when capacity is exceeded.

Q314. Standard data structures for $O(1)$ LRU operations?

A: Hash map plus doubly linked list.

Q315. What is LFU Cache?

A: A cache that evicts the least frequently used item, with a recency tie-breaker in common implementations.

13. Stack

Q316. What is a stack?

A: A LIFO data structure: last in, first out.

Q317. Common stack operations?

A: Push, pop, top/peek, and often empty/size.

Q318. What is stack overflow?

A: Attempting to push beyond a fixed stack capacity, or exhausting the call stack through excessive recursion.

Q319. How do you reverse a stack recursively?

A: Pop all elements recursively, then insert each popped element at the bottom.

Q320. How do you sort a stack using recursion?

A: Recursively remove the top, sort the remainder, then insert the removed element in sorted position.

Q321. How do you check balanced parentheses?

A: Push opening brackets and match each closing bracket against the top.

Q322. What is postfix evaluation?

A: Scan tokens; push operands and apply an operator to the required number of top operands.

Q323. Why is postfix evaluation simple?

A: Operator precedence and parentheses are already encoded by token order.

Q324. What is infix notation?

A: Operators appear between operands, such as $a+b$.

Q325. What is postfix notation?

A: Operators appear after operands, such as $ab+$.

Q326. How do you convert infix to postfix?

A: Use an operator stack and precedence/associativity rules.

Q327. What are redundant brackets?

A: Parentheses that do not affect the expression's evaluation or grouping.

Q328. What is the longest valid parentheses problem?

A: Find the longest contiguous substring containing balanced parentheses.

Q329. How can longest valid parentheses be solved?

A: Use a stack of indices or a two-pass counter method.

Q330. What is the score of balanced parentheses?

A: Compute nested/concatenated pair contributions using a stack or depth-based rule.

Q331. How do you implement a stack using queues?

A: Rearrange queue elements on push or pop depending on the chosen design.

Q332. How can two stacks share one array?

A: Grow one from the beginning and the other from the end.

Q333. How can k stacks share one array efficiently?

A: Maintain arrays for values, next links, stack tops, and a free-list of available positions.

Q334. How can getMin() be $O(1)$?

A: Maintain auxiliary minimum information or encode values so the minimum can be recovered without scanning.

Q335. What is a monotonic stack?

A: A stack maintained in increasing or decreasing order to answer nearest/next greater/smaller relationships efficiently.

Q336. What is Next Greater Element?

A: For each element, find the first element to its right that is greater.

Q337. Complexity of monotonic-stack NGE?

A: $O(n)$ because each element is pushed and popped at most once.

Q338. What is Stock Span?

A: Number of consecutive previous days whose price is $\leq$ today's price.

Q339. What is Previous Smaller Element?

A: The nearest smaller element to the left of each element.

Q340. How is Largest Rectangle in Histogram solved?

A: Use a monotonic increasing stack to determine each bar's maximal width.

Q341. How do you find the largest rectangle of 1s in a binary matrix?

A: Convert each row into histogram heights and apply largest-rectangle-in-histogram.

Q342. What is Trapping Rain Water?

A: Compute water trapped between elevation bars; two pointers or prefix maxima solve it in $O(n)$.

Q343. What is the remove-k-digits problem?

A: Remove k digits to produce the smallest possible number, commonly using a monotonic increasing stack.

Q344. Why does the remove-k-digits stack pop larger preceding digits?

A: Moving a smaller digit earlier produces a smaller lexicographic/numeric result.

Q345. What is Decode String?

A: Decode nested repetition expressions such as $3[a2[c]]$ using stacks or recursion.

14. Queue & Deque

Q346. What is a queue?

A: A FIFO data structure: first in, first out.

Q347. Basic queue operations?

A: Enqueue, dequeue, front/peek, and empty/size.

Q348. What is a circular queue?

A: A queue whose storage wraps around to reuse freed positions.

Q349. Why use a circular queue?

A: It avoids wasting array space after dequeues.

Q350. How can a queue be implemented using two stacks?

A: Use one stack for incoming elements and one for outgoing elements; transfer only when the outgoing stack is empty.

Q351. Amortized complexity of queue using two stacks?

A: $O(1)$ amortized per operation.

Q352. What is FIFO page replacement?

A: Evict the page that has been in memory the longest.

Q353. How do you reverse a queue?

A: Use a stack or recursion.

Q354. How do you reverse the first k queue elements?

A: Move the first k elements to a stack, push them back, then rotate the remaining elements appropriately.

Q355. How do you interleave two halves of a queue?

A: Split into halves and alternately enqueue one element from each half.

Q356. What is a deque?

A: A double-ended queue supporting insertion and deletion at both ends.

Q357. What is a deque useful for?

A: Sliding-window maximum, monotonic queues, palindrome checks, and double-ended scheduling.

Q358. How do you implement a deque with a circular array?

A: Maintain front/rear indices modulo capacity.

Q359. What is a monotonic deque?

A: A deque that maintains elements in sorted order of value to expose extrema efficiently.

Q360. What is the complexity of sliding-window maximum using a deque?

A: $O(n)$ time and $O(k)$ auxiliary space.

Q361. What is the longest subarray with $\max - \min \leq X$ pattern?

A: Maintain two monotonic deques: one for maximums and one for minimums.

Q362. Why use two deques for max-min constraints?

A: They provide $O(1)$ access to the current maximum and minimum while the window changes.

15. Binary Trees

Q363. What is a binary tree?

A: A tree where each node has at most two children.

Q364. What is inorder traversal?

A: Left → Root → Right.

Q365. What is preorder traversal?

A: Root → Left → Right.

Q366. What is postorder traversal?

A: Left → Right → Root.

Q367. What is level-order traversal?

A: Visit nodes level by level, typically using a queue.

Q368. What is the size of a binary tree?

A: The total number of nodes.

Q369. How do you compute tree height?

A: $1 + \max(\text{height}(\text{left}), \text{height}(\text{right}))$, with the base convention defined by the problem.

Q370. How do you compute maximum width?

A: Process each level and count its nodes, or use indexed positions when the definition requires structural width.

Q371. What is a balanced binary tree?

A: A tree where the height difference between left and right subtrees satisfies the specified bound at every node.

Q372. How can balance be checked efficiently?

A: Return subtree height and propagate a failure marker when imbalance is detected.

Q373. How do you check whether two trees are identical?

A: Their corresponding nodes must have equal values and identical null/non-null structure.

Q374. What does it mean for two trees to be mirrors?

A: The left subtree of one corresponds to the right subtree of the other recursively.

Q375. What is a sum tree?

A: Each non-leaf node equals the sum of values in its left and right subtrees, according to the problem's convention.

Q376. What is a left view?

A: The first visible node at every depth when viewing the tree from the left.

Q377. How can left view be obtained?

A: BFS by recording the first node of each level, or DFS prioritizing left children.

Q378. What is right view?

A: The first visible node at every depth from the right side.

Q379. What is root-to-leaf path sum?

A: Determine whether a root-to-leaf path has the required total sum.

Q380. What is maximum path sum?

A: Maximum sum of a path between any two nodes, where the path need not pass through the root.

Q381. What is serialization?

A: Converting a tree into a storable/transmittable sequence that preserves enough information to reconstruct it.

Q382. Why are null markers important in serialization?

A: They preserve tree structure and distinguish different shapes with identical values.

Q383. How do you perform iterative inorder traversal?

A: Push the entire left chain onto a stack, process a node, then move to its right subtree.

Q384. How do you perform iterative preorder?

A: Use a stack, process root, then push right and left children.

Q385. How do you perform iterative postorder?

A: Use two stacks or one stack with state/last-visited tracking.

Q386. What is Morris traversal?

A: A traversal that temporarily creates threaded links to achieve $O(1)$ auxiliary space.

Q387. What is tree diameter?

A: The longest path between two nodes, measured according to the problem's edge/node convention.

Q388. How can diameter be computed in $O(n)$?

A: Compute subtree heights once and update the best path through each node.

Q389. How do you construct a tree from preorder and inorder?

A: Preorder identifies the root; inorder identifies left/right partitions. A value-to-index map makes partition lookup $O(1)$.

Q390. What is boundary traversal?

A: Visit root, left boundary, leaves, and reversed right boundary while avoiding duplicates.

Q391. What is vertical traversal?

A: Group nodes by horizontal distance, with ordering rules determined by the problem.

Q392. What is top view?

A: The first visible node at each horizontal distance from the root.

Q393. What is bottom view?

A: The last visible node at each horizontal distance under the specified traversal ordering.

Q394. What is diagonal traversal?

A: Group nodes along diagonal directions, commonly using a queue/map or recursive approach.

Q395. What is LCA in a binary tree?

A: The lowest node that is an ancestor of both target nodes.

Q396. What is the distance-K problem?

A: Find all nodes exactly K edges away from a target, including nodes through parent connections.

Q397. How can a tree be flattened to a linked list?

A: Rewire nodes in preorder order so each node's right pointer becomes the next node and left pointers become null.

Q398. What is a subtree check?

A: Determine whether one tree occurs as a structurally/value-equivalent subtree of another.

16. Binary Search Tree

Q399. What is a BST?

A: A binary tree where values in the left subtree are ordered before the node and values in the right subtree after it, subject to the duplicate convention.

Q400. What is the average search complexity in a balanced BST?

A: $O(\log n)$.

Q401. What is the worst-case BST search complexity?

A: $O(n)$ when the tree degenerates into a chain.

Q402. Why is inorder traversal important for BSTs?

A: It produces keys in sorted order.

Q403. How do you insert into a BST?

A: Compare the key with the current node and recursively/iteratively descend left or right until an empty position is found.

Q404. What are the three deletion cases in a BST?

A: Leaf, one child, and two children.

Q405. How do you delete a BST node with two children?

A: Replace it with its inorder successor/predecessor and then delete that replacement node.

Q406. What is BST floor?

A: The greatest key less than or equal to the target.

Q407. What is BST ceil?

A: The smallest key greater than or equal to the target.

Q408. What is inorder predecessor?

A: The largest key smaller than the target node.

Q409. What is inorder successor?

A: The smallest key larger than the target node.

Q410. How do you find kth smallest in a BST?

A: Perform inorder traversal and stop after visiting k nodes.

Q411. How can kth-smallest space be optimized?

A: Use iterative traversal or Morris traversal depending on constraints.

Q412. How do you solve Two Sum in a BST?

A: Use two BST iterators, one ascending and one descending, or flatten to sorted values.

Q413. How do you build a balanced BST from a sorted array?

A: Choose the middle element as root recursively.

Q414. How do you merge two BSTs?

A: Inorder-traverse both trees and merge the resulting sorted sequences, or use simultaneous iterators.

Q415. How do you validate a BST?

A: Carry valid lower/upper bounds or verify that inorder traversal is strictly ordered under the duplicate convention.

Q416. How is LCA found in a BST?

A: If both keys are smaller go left; if both larger go right; otherwise the current node is the split point.

Q417. What is Recover BST?

A: Restore a BST where two node values have been swapped.

Q418. How are swapped BST nodes detected?

A: Find inorder ordering violations and identify the misplaced nodes.

Q419. What is the largest BST subtree problem?

A: Find the largest subtree that itself satisfies BST ordering.

Q420. What information should a largest-BST-subtree DP return?

A: Whether the subtree is BST, its size, minimum, maximum, and the best BST size as needed.

17. Heap / Priority Queue

Q421. What is a heap?

A: A complete binary tree satisfying a heap-order property.

Q422. What is a min-heap?

A: Every parent is $\leq$ its children.

Q423. What is a max-heap?

A: Every parent is $\geq$ its children.

Q424. How is a heap commonly represented?

A: In an array without explicit child pointers.

Q425. For zero-based indexing, what are a node's children?

A: Left = $2i+1$, right = $2i+2$.

Q426. What is heap height?

A: $O(\log n)$.

Q427. Complexity of heap insertion?

A: $O(\log n)$.

Q428. Complexity of removing the root?

A: $O(\log n)$.

Q429. Complexity of peeking at the root?

A: $O(1)$.

Q430. What is Heap Sort complexity?

A: $O(n \log n)$ time and $O(1)$ auxiliary array space for in-place variants.

Q431. What is build-heap complexity?

A: $O(n)$, not $O(n \log n)$.

Q432. How do you find kth largest efficiently?

A: Maintain a min-heap of size k.

Q433. Complexity of kth-largest using a size-k heap?

A: $O(n \log k)$.

Q434. How do you find k largest elements?

A: Use a min-heap of size k or sort depending on constraints.

Q435. What is the connect-ropes problem?

A: Repeatedly connect the two shortest ropes; a min-heap gives the optimal greedy choice.

Q436. Why is a min-heap correct for connecting ropes?

A: Combining the two smallest costs minimizes their repeated contribution to the total cost.

Q437. What is a nearly sorted array?

A: Each element is at most k positions away from its sorted position.

Q438. How do you sort a nearly sorted array?

A: Maintain a min-heap of up to k+1 elements.

Q439. How do you merge k sorted arrays?

A: Put the first element from each array into a min-heap and repeatedly extract the minimum.

Q440. Complexity of merging k sorted arrays?

A: $O(N \log k)$.

Q441. How do you find median of a data stream?

A: Use two heaps: max-heap for lower half and min-heap for upper half.

Q442. How do you maintain heap balance for streaming median?

A: Keep sizes equal or differ by at most one and ensure every lower-half value $\leq$ every upper-half value.

Q443. How do you find kth largest in a stream?

A: Maintain a min-heap of size k.

Q444. What is the smallest range covering k sorted lists?

A: Maintain one current element from each list in a min-heap while tracking the current maximum.

Q445. What is the Skyline problem?

A: Compute visible building boundaries; sweep-line plus heap/multiset is a common approach.

Q446. How can elements be sorted by frequency?

A: Count frequencies, then sort/group according to frequency and tie-breaking rules.

18. Graphs

Q447. What is a graph?

A: A collection of vertices and edges representing relationships.

Q448. What is a directed graph?

A: Edges have direction.

Q449. What is an undirected graph?

A: Edges represent bidirectional relationships.

Q450. What is a weighted graph?

A: Edges have associated costs/weights.

Q451. What is an adjacency matrix?

A: A $V \times V$ matrix indicating whether an edge exists, optionally storing its weight.

Q452. What is an adjacency list?

A: For each vertex, store a list of its neighboring vertices/edges.

Q453. When is an adjacency list preferable?

A: For sparse graphs, because it uses $O(V+E)$ storage.

Q454. What is BFS?

A: Breadth-first search explores vertices level by level using a queue.

Q455. What is DFS?

A: Depth-first search explores as far as possible along a branch before backtracking.

Q456. BFS complexity?

A: $O(V+E)$ with adjacency lists.

Q457. DFS complexity?

A: $O(V+E)$ with adjacency lists.

Q458. Which traversal finds shortest paths in an unweighted graph?

A: BFS.

Q459. What is flood fill?

A: Starting from a cell, visit connected cells satisfying the same region condition and replace their values.

Q460. How do you count islands?

A: Treat land cells as graph nodes and run BFS/DFS from each unvisited land cell.

Q461. What are connected components?

A: Maximal sets of vertices connected through paths in an undirected graph.

Q462. How do you detect a cycle in an undirected graph with DFS?

A: Track the parent; an already visited neighbor that is not the parent indicates a cycle.

Q463. How do you detect a cycle in a directed graph?

A: Use DFS recursion-state tracking or Kahn's topological-sort method.

Q464. What is a bipartite graph?

A: A graph whose vertices can be divided into two sets with no edge inside the same set.

Q465. How do you test bipartiteness?

A: BFS/DFS two-coloring; a conflict means the graph is not bipartite.

Q466. What is Rotten Oranges?

A: A multi-source BFS problem where initially rotten cells simultaneously spread to adjacent fresh cells.

Q467. Why is multi-source BFS useful for Rotten Oranges?

A: All sources start at distance/time zero, so BFS naturally computes minimum infection time.

Q468. What is topological sorting?

A: An ordering of vertices in a directed acyclic graph such that every directed edge $u \rightarrow v$ places u before v .

Q469. Can a graph with a directed cycle have a topological ordering?

A: No.

Q470. What is Kahn's algorithm?

A: Repeatedly remove zero-indegree vertices and reduce indegrees of their outgoing neighbors.

Q471. What is DFS-based topological sorting?

A: Visit descendants first and push a node after processing all outgoing edges.

Q472. What is Course Schedule I?

A: Determine whether prerequisite dependencies form a feasible DAG.

Q473. What is Course Schedule II?

A: Produce a valid course ordering if the dependency graph is acyclic.

Q474. What is Alien Dictionary?

A: Infer character precedence from an ordered list of alien words and topologically sort the resulting constraints.

Q475. What is Dijkstra's algorithm?

A: A greedy shortest-path algorithm for graphs with nonnegative edge weights.

Q476. Complexity of Dijkstra with a binary heap?

A: Commonly $O((V+E) \log V)$, depending on implementation.

Q477. Why can't standard Dijkstra handle negative edges?

A: A finalized shortest distance can later be improved by a negative edge, violating its greedy assumption.

Q478. What is Bellman-Ford?

A: A shortest-path algorithm that relaxes all edges repeatedly and can detect reachable negative cycles.

Q479. Complexity of Bellman-Ford?

A: $O(VE)$.

Q480. What is Floyd-Warshall?

A: A dynamic-programming algorithm for all-pairs shortest paths.

Q481. Complexity of Floyd-Warshall?

A: $O(V^3)$ time and $O(V^2)$ space.

Q482. What is a multi-source BFS?

A: BFS initialized simultaneously with multiple source vertices.

Q483. What is Word Ladder?

A: Find the shortest sequence of one-character transformations from one word to another through valid dictionary words.

Q484. Why is BFS suitable for Word Ladder?

A: Each transformation has equal cost, so BFS finds the minimum number of transformations.

Q485. What is Prim's algorithm?

A: A greedy algorithm that grows a minimum spanning tree by repeatedly adding the cheapest edge connecting the tree to an outside vertex.

Q486. What is Kruskal's algorithm?

A: Sort edges by weight and add them if they connect different components.

Q487. What data structure is commonly used by Kruskal?

A: Disjoint Set Union (DSU/Union-Find).

Q488. What is DSU?

A: A structure supporting efficient set merging and representative queries.

Q489. What are path compression and union by rank/size?

A: Optimizations that keep DSU trees shallow.

Q490. What is the approximate amortized complexity of optimized DSU?

A: $O(\alpha(n))$ per operation, effectively constant for practical input sizes.

Q491. How does DSU detect an undirected cycle?

A: If an edge connects two vertices already in the same set, it creates a cycle.

Q492. What is an MST?

A: A minimum-weight set of edges connecting all vertices of a connected weighted undirected graph without cycles.

Q493. How many edges does an MST contain?

A: $V-1$ for a connected graph with V vertices.

Q494. Can an MST be unique?

A: Yes; distinct edge weights guarantee uniqueness, while equal weights can permit multiple MSTs.

Q495. What is the Matrix Tree Theorem used for?

A: Counting spanning trees of a graph using a determinant of a reduced Laplacian matrix.

Q496. How can connected components be counted using DSU?

A: Start with each vertex separate and union endpoints of every edge; count final distinct representatives.

Q497. What is job sequencing with deadlines?

A: Schedule profitable jobs before deadlines to maximize total profit, commonly using greedy ordering and slot management.

Q498. What is shortest cycle in an unweighted undirected graph?

A: BFS from each vertex can determine the shortest cycle through that source.

Q499. What is a curved-edge shortest-path problem?

A: Consider paths that may use a special curved edge, often by computing shortest distances from both endpoints and evaluating candidate paths.

Q500. What is minimum-cost path in a grid?

A: Model cells as graph states and use an appropriate shortest-path algorithm based on movement costs.

19. Greedy Algorithms

Q501. What is a greedy algorithm?

A: It repeatedly makes the locally best choice with the expectation that these choices produce a global optimum.

Q502. What must be established for greedy correctness?

A: Typically a greedy-choice property and optimal substructure, often proven using an exchange argument.

Q503. What is Fractional Knapsack?

A: Items may be divided, so select by highest value-to-weight ratio.

Q504. Why does greedy work for fractional but not general 0/1 knapsack?

A: Divisibility makes the highest ratio choice safe in fractional knapsack; indivisible items can make locally best ratios globally suboptimal.

Q505. What is the cookie-assignment strategy?

A: Sort children and cookies and greedily assign the smallest cookie capable of satisfying the current smallest requirement.

Q506. What is activity selection?

A: Select the maximum number of non-overlapping activities.

Q507. Which greedy rule solves activity selection?

A: Choose the activity with the earliest finishing time.

Q508. Why earliest finish works?

A: It leaves the maximum remaining time for future activities.

Q509. What is Huffman coding?

A: A greedy algorithm that repeatedly combines the two least frequent symbols to build an optimal prefix code.

Q510. What data structure is ideal for Huffman coding?

A: A min-heap.

Q511. What is Jump Game?

A: Determine whether the last index can be reached from allowed jump lengths.

Q512. What is the greedy idea for Jump Game?

A: Track the farthest reachable index while scanning.

Q513. What is minimum jumps to reach the end?

A: Treat the current reachable range as a level and greedily choose the next range with the farthest reach.

Q514. What is job sequencing?

A: Schedule jobs with deadlines and profits to maximize total profit under slot constraints.

Q515. What is the greedy order for job sequencing?

A: Sort jobs by descending profit and place each job in the latest available slot before its deadline.

Q516. What is a non-overlapping interval problem?

A: Remove the minimum number of intervals so the remaining intervals do not overlap.

Q517. What greedy strategy solves interval removal?

A: Keep the interval with the earliest finish time whenever an overlap occurs.

Q518. What is the Gas Station problem?

A: Determine a starting station allowing a complete circular tour given gas and travel costs.

Q519. Key greedy insight for Gas Station?

A: If total gas is at least total cost, a solution exists; when the running balance becomes negative, the next station becomes the new candidate.

Q520. What is Stable Marriage?

A: Match two groups so no pair prefers each other over their assigned partners.

Q521. What algorithm solves Stable Marriage?

A: Gale-Shapley deferred acceptance algorithm.

Q522. What is the key guarantee of Gale-Shapley?

A: It produces a stable matching and is optimal for the proposing side under the standard formulation.

Q523. What is minimum cost to cut a board?

A: Choose cut ordering strategically; a greedy approach can be applied when costs correspond to current segment dimensions.

Q524. What is the "no two adjacent same" rearrangement problem?

A: Rearrange characters so identical characters are not adjacent, commonly using a max-heap.

Q525. What is minimize cash flow?

A: Simplify debts among people by repeatedly settling the largest creditor/debtor balances.

Q526. What is minimizing maximum height difference?

A: Sort heights and decide whether each height can be increased/decreased under the allowed operation to minimize the final range.

20. Dynamic Programming

Q527. What is dynamic programming?

A: Solve overlapping subproblems once and reuse their results while building an optimal/counting solution.

Q528. What are the two key properties of DP?

A: Overlapping subproblems and optimal substructure.

Q529. What is memoization?

A: Top-down recursion with caching of computed states.

Q530. What is tabulation?

A: Bottom-up computation of DP states in dependency order.

Q531. What is state in DP?

A: The minimal information needed to uniquely describe a subproblem.

Q532. What is the first step in designing a DP?

A: Define the state precisely.

Q533. What is Climbing Stairs?

A: Count ways to reach step n when each move can advance according to the allowed step sizes.

Q534. What recurrence appears in standard Climbing Stairs?

A: $dp[i] = dp[i-1] + dp[i-2]$.

Q535. How can Climbing Stairs space be optimized?

A: Keep only the previous two states.

Q536. What is the no-consecutive-1s binary-string problem?

A: Count binary strings of length n without adjacent 1s.

Q537. What is Weighted Climbing Stairs?

A: Minimize cost to reach the top by choosing allowed step sizes with associated landing costs.

Q538. What is Frog Jump?

A: Find the minimum cost to reach a target stair/cell under allowed jumps and movement costs.

Q539. What is House Robber?

A: Maximize stolen value without selecting adjacent houses.

Q540. House Robber recurrence?

A: $dp[i] = \max(dp[i-1], dp[i-2] + value[i])$.

Q541. What is Coin Change?

A: Determine the minimum number of coins needed to make a target amount, or report impossibility.

Q542. What is Coin Change II?

A: Count the number of combinations that make a target amount.

Q543. Why does loop order matter in coin-change counting?

A: It determines whether combinations are counted once or different orderings are treated as distinct.

Q544. What is decoding count?

A: Count ways to map digit strings to letters under valid single- and double-digit encoding rules.

Q545. What is Derangements?

A: Count permutations where no element remains in its original position.

Q546. Derangement recurrence?

A: $D(n) = (n-1) (D(n-1) + D(n-2))$.

Q547. What is Unique Paths?

A: Count paths from top-left to bottom-right using permitted moves.

Q548. What is LCS?

A: Longest Common Subsequence between two sequences.

Q549. LCS recurrence?

A: If characters match, $1 + dp[i-1][j-1]$; otherwise take $\max(dp[i-1][j], dp[i][j-1])$.

Q550. Complexity of standard LCS DP?

A: $O(mn)$ time and $O(mn)$ space, reducible to $O(\min(m,n))$ for length-only computation.

Q551. What is Edit Distance?

A: Minimum insertions, deletions, and substitutions needed to transform one string into another.

Q552. What is Distinct Subsequences?

A: Count how many subsequences of one string equal another string.

Q553. What is 0/1 Knapsack?

A: Maximize value subject to capacity when each item can be selected at most once.

Q554. What is the 0/1 Knapsack recurrence?

A: Either skip the item or take it if capacity permits: $\max(\text{skip}, \text{take})$.

Q555. What is subset sum?

A: Determine whether some subset of values sums to a target.

Q556. What is partition equal subset sum?

A: Determine whether an array can be split into two subsets with equal sum; target becomes $\text{total}/2$ if total is even.

Q557. What is target sum DP?

A: Assign plus/minus signs to achieve a target; it can often be transformed into a subset-sum counting problem.

Q558. What is Rod Cutting?

A: Maximize revenue by cutting a rod into pieces with known prices.

Q559. What is the key difference between 0/1 and unbounded knapsack?

A: 0/1 allows each item once; unbounded allows repeated selection.

Q560. What is Unique Paths with obstacles?

A: Count paths while treating blocked cells as states with zero ways.

Q561. What is maximum path sum in a matrix?

A: DP each cell from allowed predecessor cells while maintaining the best achievable sum.

Q562. What is Longest Increasing Path in a matrix?

A: Treat increasing moves as a DAG and use DFS memoization or sorted DP.

Q563. What is Longest Common Substring?

A: Longest contiguous sequence common to both strings.

Q564. Difference between LCS and Longest Common Substring?

A: LCS need not be contiguous; common substring must be contiguous.

Q565. What is Shortest Common Supersequence?

A: The shortest string containing both input strings as subsequences.

Q566. How is SCS length related to LCS?

A: $m+n-LCS$.

Q567. What is Interleaving String?

A: Determine whether a string can be formed by interleaving two strings while preserving the order of each.

Q568. What is wildcard matching?

A: Match a text against a pattern containing wildcard symbols such as `?` and `*`, according to the problem's semantics.

Q569. What is Longest Palindromic Substring?

A: Find the longest contiguous substring that is a palindrome.

Q570. How can stock with cooldown be modeled?

A: Use states such as holding, sold, and cooldown/rest.

Q571. How do you model stock with at most two transactions?

A: Use transaction-count states or maintain four buy/sell states.

Q572. How does stock with k transactions generalize?

A: DP over day and transaction count, with optimized state transitions.

Q573. What is stock with transaction fee?

A: Subtract the transaction fee during a buy/sell transition according to the chosen state formulation.

Q574. What is LIS?

A: Longest Increasing Subsequence.

Q575. Complexity of classic $O(n^2)$ LIS DP?

A: $O(n^2)$ time.

Q576. Complexity of patience-sorting-style LIS?

A: $O(n \log n)$ for length computation.

Q577. What is Number of LIS?

A: Count how many increasing subsequences attain the maximum LIS length.

Q578. What is Longest Bitonic Subsequence?

A: A subsequence that first increases and then decreases.

Q579. What is Largest Divisible Subset?

A: Find the largest subset where for every pair, one number divides the other under the problem's positive-number assumptions.

Q580. What is Longest String Chain?

A: Build a chain where each word can be obtained from the next longer word by deleting one character.

Q581. What is Maximum Envelopes?

A: Maximum nesting of envelopes; sort one dimension and apply LIS on the other with an appropriate tie rule.

Q582. What is Matrix Chain Multiplication?

A: Find the parenthesization minimizing scalar multiplication cost.

Q583. What is the MCM DP state?

A: Minimum multiplication cost for multiplying matrices i through j .

Q584. What is palindrome partitioning DP?

A: Minimize cuts or enumerate partitions where every selected substring is a palindrome.

Q585. What is Boolean Parenthesization?

A: Count ways to parenthesize a boolean expression so it evaluates to the target value.

Q586. What is Partition Array for Maximum Sum?

A: Partition into bounded-length groups and maximize the sum of each group's maximum multiplied by its length.

Q587. What is Egg Dropping?

A: Minimize the worst-case number of trials needed to determine a critical floor with a limited number of eggs.

Q588. What is Optimal Strategy for a Game?

A: Two players alternately choose from ends; DP computes the best guaranteed score under optimal play.

21. Number Theory

Q589. What is nCr ?

A: Number of ways to choose r items from n : $n! / (r! (n-r)!)$.

Q590. Why avoid direct factorial computation for large nCr ?

A: Factorials overflow quickly; multiplicative, DP, modular, or prime-factor methods are safer depending on constraints.

Q591. What is Euler's Totient $\phi(n)$?

A: Number of integers from 1 to n that are coprime with n .

Q592. Formula for Euler's Totient?

A: $\phi(n) = n \prod (1 - 1/p)$ over distinct prime factors p of n .

Q593. What is the Sieve of Eratosthenes?

A: An efficient method for finding all primes up to N by repeatedly marking multiples of each discovered prime.

Q594. Complexity of the standard sieve?

A: $O(N \log \log N)$ time and $O(N)$ space.

Q595. What is Pascal's Triangle?

A: A triangular arrangement where each interior value is the sum of the two values above it.

Q596. What combinatorial property does Pascal's Triangle encode?

A: Binomial coefficients.

Q597. What is modular exponentiation?

A: Compute $a^b \bmod m$ efficiently by exponentiation by squaring while reducing modulo m at each step.

Q598. Complexity of modular exponentiation?

A: $O(\log b)$.

Q599. What is a segmented sieve?

A: A sieve applied to a range $[L, R]$ using primes up to $\sqrt{R}$, useful when R is large but the interval is manageable.

Q600. Why use a segmented sieve?

A: It avoids allocating an array of size R when only a smaller interval needs to be examined.

22. Trie

Q601. What is a Trie?

A: A prefix tree where paths represent strings character by character.

Q602. What operations are naturally efficient in a Trie?

A: Insert, exact search, prefix search, and prefix enumeration.

Q603. Complexity of Trie insertion?

A: $O(L)$, where L is the key length.

Q604. Complexity of Trie search?

A: $O(L)$.

Q605. What is an end-of-word marker?

A: A flag indicating that a complete stored word terminates at a node.

Q606. Why is an end-of-word marker necessary?

A: Because one word can be a prefix of another.

Q607. What is the longest word with all prefixes?

A: Find the longest word for which every prefix is itself a stored word.

Q608. How can a Trie solve autocomplete?

A: Follow the query prefix to a node, then traverse its descendants to generate suggestions.

Q609. What is a distinct-substring Trie approach?

A: Insert every suffix and count newly created Trie nodes/paths according to the representation.

Q610. What is a binary Trie?

A: A Trie whose branches represent bits, commonly used for XOR optimization.

Q611. How does a binary Trie maximize XOR?

A: At each bit, prefer the opposite bit to the query bit because it contributes 1 to XOR.

Q612. Complexity of maximum XOR using binary Trie?

A: $O(n \cdot B)$, where B is the number of relevant bits.

Q613. What are XOR queries with an upper bound?

A: Query maximum XOR against values satisfying an additional constraint such as $\text{value} \leq m$.

Q614. How can binary Trie queries be constrained by a maximum value?

A: Insert only eligible values offline or maintain a data structure aligned with query thresholds.

Q615. How can duplicate binary matrix rows be detected?

A: Represent each row in a Trie or use hashing; identical paths identify duplicates.

Q616. What is the "XOR less than K" Trie pattern?

A: Traverse bits while counting branches that keep the XOR prefix below K , then continue along the constrained branch.

Q617. Why does bitwise greedy work in maximum XOR?

A: Higher-order bits dominate the numeric value, so maximizing them first gives the largest XOR.

Q618. What is palindrome-pair search with Trie?

A: Store words/reversed words with palindrome metadata so complementary prefix/suffix conditions can be queried efficiently.

23. String Matching

Q619. What is substring search?

A: Determine whether a pattern occurs contiguously within a text.

Q620. What is the naive substring-search complexity?

A: $O(nm)$ worst case for text length n and pattern length m .

Q621. What is Rabin-Karp?

A: A rolling-hash pattern-matching algorithm that compares hash values and verifies matches when hashes agree.

Q622. Average complexity of Rabin-Karp?

A: Typically near $O(n+m)$ with a good hash, but worst case can degrade due to collisions.

Q623. What is KMP?

A: Knuth-Morris-Pratt pattern matching avoids rechecking characters by using prefix-function/LPS information.

Q624. What is LPS?

A: Longest Proper Prefix that is also a Suffix.

Q625. Complexity of KMP?

A: $O(n+m)$.

Q626. What is the Z algorithm?

A: It computes for every position the length of the longest substring starting there that matches the prefix of the string.

Q627. How can Z algorithm perform pattern matching?

A: Build `pattern + delimiter + text` and find positions where Z equals the pattern length.

Q628. What is Manacher's algorithm?

A: A linear-time algorithm for finding the longest palindromic substring.

Q629. Complexity of Manacher's algorithm?

A: $O(n)$.

Q630. What is the key idea behind Manacher?

A: Reuse palindrome-radius information from mirrored centers to avoid redundant expansion.

Q631. How can a longest palindromic substring also be solved without Manacher?

A: Expand around centers or use dynamic programming, with higher worst-case complexity.

Q632. What is palindrome substring query preprocessing?

A: Precompute information that answers whether arbitrary substrings are palindromes efficiently.

Q633. What is the repeated-string matching problem?

A: Determine the minimum number of repetitions of A required for B to occur as a substring.

Q634. Why is a repeated-string search bounded?

A: B can only require around $\text{ceil}(|B|/|A|)$ repetitions plus a small constant number for boundary alignment.

Q635. How do you search one array as a subarray of another?

A: Use naive matching, hashing, KMP-style matching, or other sequence-search methods depending on constraints.

24. Range Queries

Q636. What is a Segment Tree?

A: A tree-based structure that stores aggregate information over array ranges.

Q637. What operations does a segment tree support?

A: Range queries and point updates in $O(\log n)$, with suitable merge operations.

Q638. What is the build complexity of a segment tree?

A: $O(n)$.

Q639. What is a range minimum query?

A: Return the minimum value in a specified interval.

Q640. Can a segment tree support maximum queries?

A: Yes, by storing the maximum for every segment.

Q641. Can a segment tree support GCD/LCM/XOR?

A: Yes, provided the operation can be represented by an appropriate merge rule; implementation details and overflow considerations vary.

Q642. What is a point update?

A: Change one array element and update all segment nodes whose ranges contain that position.

Q643. What is lazy propagation?

A: Delay range updates by storing pending changes on tree nodes until their values are needed.

Q644. Why is lazy propagation useful?

A: It reduces range-update complexity from potentially $O(n)$ to $O(\log n)$ per operation for supported update/query types.

Q645. What is range sign flipping?

A: Apply a sign change to all values in a range; the segment-tree node must store enough information to update aggregates correctly.

Q646. What is a Fenwick Tree?

A: A Binary Indexed Tree supporting prefix aggregation and point updates efficiently.

Q647. Complexity of Fenwick update/query?

A: $O(\log n)$.

Q648. What is Fenwick Tree especially good for?

A: Prefix sums, point updates, frequency counting, inversion-related tasks, and coordinate-compressed indexes.

Q649. Segment Tree vs Fenwick Tree?

A: Fenwick is simpler and memory-efficient for prefix-invertible aggregates such as sums; Segment Tree supports more general range operations and richer queries.

Q650. Can Fenwick Tree support range updates?

A: Yes, with appropriate difference-array or multiple-tree techniques depending on the required query/update

combination.

Q651. What is a 3D Fenwick Tree?

A: A multidimensional extension supporting point updates and prefix aggregates in three dimensions.

Q652. What is a common complexity for a 3D Fenwick operation?

A: $O(\log^3 n)$, depending on dimensions and implementation.

Q653. What is a range maximum of fixed-size subarrays?

A: Find the maximum for every window of size k ; a deque, heap, or segment tree can solve it depending on constraints.

Q654. When is a Segment Tree preferable to a sparse table?

A: When updates are required; sparse tables are typically designed for static data.

25. Rapid Complexity Recall

Q655. Complexity of array indexed access?

A: $O(1)$.

Q656. Complexity of linear search?

A: $O(n)$.

Q657. Complexity of binary search?

A: $O(\log n)$.

Q658. Complexity of Merge Sort?

A: $O(n \log n)$.

Q659. Average complexity of Quick Sort?

A: $O(n \log n)$.

Q660. Worst-case Quick Sort?

A: $O(n^2)$.

Q661. Heap insertion?

A: $O(\log n)$.

Q662. Heap extraction?

A: $O(\log n)$.

Q663. Heap peek?

A: $O(1)$.

Q664. Hash lookup average?

A: $O(1)$ expected, with worst-case behavior depending on implementation.

Q665. BFS/DFS with adjacency list?

A: $O(V+E)$.

Q666. Dijkstra with binary heap?

A: Commonly $O((V+E)\log V)$.

Q667. Bellman-Ford?

A: $O(VE)$.

Q668. Floyd-Warshall?

A: $O(V^3)$.

Q669. DSU optimized operation?

A: $O(\alpha(n))$ amortized.

Q670. Standard LCS?

A: $O(mn)$.

Q671. Standard edit distance?

A: $O(mn)$.

Q672. LIS with binary-search tails?

A: $O(n \log n)$.

Q673. KMP?

A: $O(n+m)$.

Q674. Z algorithm?

A: $O(n)$.

Q675. Manacher?

A: $O(n)$.

Q676. Trie operation?

A: $O(L)$, where L is key length.

Q677. Segment Tree query/update?

A: $O(\log n)$.

Q678. Fenwick query/update?

A: $O(\log n)$.

Q679. Sliding window typical complexity?

A: $O(n)$, when each element enters and leaves the window a bounded number of times.

Q680. Two-pointer typical complexity?

A: $O(n)$ after any required sorting; sorting can make the total $O(n \log n)$.

26. Rapid Pattern Recognition

Q681. Sorted array + target search?

A: Think Binary Search.

Q682. Sorted array + pair sum?

A: Think Two Pointers.

Q683. Contiguous subarray + fixed k ?

A: Think Fixed Sliding Window.

Q684. Contiguous substring + "longest" + constraint?

A: Think Variable Sliding Window.

Q685. Subarray sum repeated queries?

A: Think Prefix Sum.

Q686. Subarray sum K with negative numbers?

A: Think Prefix Sum + Hash Map.

Q687. Subarray XOR K?

A: Think Prefix XOR + Hash Map.

Q688. Nearest greater/smaller element?

A: Think Monotonic Stack.

Q689. Maximum/minimum in every window?

A: Think Monotonic Deque.

Q690. Top K elements?

A: Think Heap/Priority Queue.

Q691. Merge K sorted structures?

A: Think Min-Heap or divide-and-conquer.

Q692. Shortest path in unweighted graph?

A: Think BFS.

Q693. Shortest path with nonnegative weights?

A: Think Dijkstra.

Q694. Negative edge weights?

A: Think Bellman-Ford or another algorithm appropriate to the graph constraints.

Q695. All-pairs shortest paths?

A: Think Floyd-Warshall or repeated single-source methods depending on graph size/density.

Q696. Dependency ordering?

A: Think Topological Sort.

Q697. Minimum spanning tree?

A: Think Prim or Kruskal.

Q698. Connectivity under unions?

A: Think DSU.

Q699. Prefix search/autocomplete?

A: Think Trie.

Q700. Maximum XOR pair?

A: Think Binary Trie.

Q701. Generate all combinations/permutations?

A: Think Backtracking.

Q702. Optimize/count overlapping subproblems?

A: Think Dynamic Programming.

Q703. Local optimal choice with interval scheduling?

A: Think Greedy, especially earliest finish time.

Q704. Range query + updates?

A: Think Segment Tree/Fenwick Tree depending on operation.

Q705. Range update + range query?

A: Think Lazy Segment Tree or an appropriate difference/prefix structure.

27. Interview Follow-Up Questions

Q706. Why is $O(n \log n)$ often preferred over $O(n^2)$?

A: Its growth is substantially slower for large n , making it more scalable.

Q707. When can an $O(n^2)$ algorithm still be acceptable?

A: When n is small enough or when the quadratic operation is simple and constraints permit it.

Q708. What should you clarify before coding?

A: Input constraints, duplicates, ordering, negative values, empty input, overflow, expected output, and whether modification of input is allowed.

Q709. Why are constraints important?

A: They determine which complexity is feasible and often reveal the intended algorithmic pattern.

Q710. Why consider integer overflow?

A: Intermediate calculations such as products, sums, midpoints, and squared values can exceed the type's range even when final answers fit.

Q711. What is an invariant?

A: A property that remains true throughout an algorithm and helps prove correctness.

Q712. What is a loop invariant?

A: A condition true before and after each iteration that explains why the algorithm remains correct.

Q713. What is amortized analysis useful for?

A: Data structures where occasional expensive operations are balanced by many cheap operations.

Q714. What is stable sorting?

A: Equal-key elements preserve their original relative order.

Q715. Why might stability matter?

A: It preserves ordering established by earlier sorting/grouping operations.

Q716. What is in-place?

A: Usually means using $O(1)$ or limited auxiliary memory beyond the input, subject to the problem's convention.

Q717. What is divide and conquer?

A: Divide the problem into smaller independent pieces, solve them recursively, and combine their results.

Q718. Examples of divide-and-conquer algorithms?

A: Merge Sort, Quick Sort, and several binary-search-based algorithms.

Q719. What is greedy vs DP?

A: Greedy commits to local choices without revisiting them; DP evaluates combinations of subproblems and stores reusable results.

Q720. What is BFS vs DFS?

A: BFS explores by distance/levels using a queue; DFS explores deeply using recursion or a stack.

Q721. When is BFS preferable?

A: Shortest path in unweighted graphs, level-order problems, and minimum-step state transitions.

Q722. When is DFS preferable?

A: Connectivity, exhaustive traversal, cycle detection, topological sorting, and many recursive tree/graph problems.

Q723. Why do many graph algorithms require a visited array?

A: To avoid repeated processing and infinite traversal through cycles.

Q724. What is state compression?

A: Representing a large state using a smaller sufficient representation, such as bit masks.

Q725. What is coordinate compression?

A: Replace large/sparse values with their rank/order positions while preserving required comparisons.

Q726. Why can sorting simplify a problem?

A: It exposes order, enables binary search, two pointers, greedy selection, and interval merging.

Q727. What is a monotonic property?

A: A property that changes only in one direction as the candidate/parameter increases or decreases.

Q728. Why is monotonicity important for binary search on answer?

A: It divides the answer space into a contiguous feasible and infeasible region.

Q729. What is an edge case?

A: An input at or near a boundary, such as empty input, one element, all equal values, all negative values, or maximum constraints.

Q730. What is the best way to communicate a coding solution in an interview?

A: State the idea, explain why it works, give complexity, mention edge cases, then code clearly.

28. Ultra-Fast One-Liners

Q731. Array random access?

A: $O(1)$.

Q732. Linked-list random access?

A: $O(n)$.

Q733. Stack order?

A: LIFO.

Q734. Queue order?

A: FIFO.

Q735. Deque supports?

A: Both ends.

Q736. BST inorder gives?

A: Sorted order.

Q737. Heap root gives?

A: Minimum for min-heap; maximum for max-heap.

Q738. BFS uses?

A: Queue.

Q739. DFS uses?

A: Stack/recursion.

Q740. Dijkstra requires?

A: Nonnegative edge weights.

Q741. Bellman-Ford supports?

A: Negative edges and negative-cycle detection.

Q742. Floyd-Warshall computes?

A: All-pairs shortest paths.

Q743. Kruskal uses?

A: Sorting + DSU.

Q744. Prim uses?

A: Greedy expansion, commonly with a priority queue.

Q745. KMP stores?

A: LPS/prefix-function information.

Q746. Manacher computes?

A: Palindrome radii.

Q747. Trie is best for?

A: Prefix operations.

Q748. Segment Tree is best for?

A: Dynamic range queries/updates.

Q749. Fenwick Tree is best for?

A: Prefix/range sums with point updates.

Q750. Boyer-Moore voting finds?

A: Majority candidate.

Q751. Kadane finds?

A: Maximum subarray sum.

Q752. Floyd's tortoise-hare finds?

A: Linked-list cycle.

Q753. Dutch National Flag sorts?

A: 0, 1, and 2.

Q754. Brian Kernighan counts?

A: Set bits.

Q755. $n \& (n-1)$ does what?

A: Clears the lowest set bit.

Q756. $n \& -n$ gives?

A: Lowest set bit.

Q757. XOR of identical values?

A: Zero.

Q758. Prefix sum duplicate means?

A: Zero-sum subarray exists between occurrences.

Q759. Prefix XOR duplicate means?

A: Zero-XOR subarray exists between occurrences.

Q760. Exact K distinct via at-most?

A: $\text{atMost}(K) - \text{atMost}(K-1)$.

Q761. LRU implementation?

A: Hash map + doubly linked list.

Q762. Median stream?

A: Two heaps.

Q763. Top K?

A: Heap.

Q764. K sorted lists?

A: Min-heap.

Q765. Dependencies?

A: Topological sort.

Q766. Connectivity unions?

A: DSU.

Q767. All-pairs shortest path?

A: Floyd-Warshall is the classic DP solution.

Q768. Minimum spanning tree?

A: Prim/Kruskal.

Q769. Overlapping intervals?

A: Sort by start/end and merge or greedily select.

Q770. All permutations?

A: Backtracking.

Q771. Repeated subproblems?

A: DP/memoization.

Q772. Locally optimal choices with proof?

A: Greedy.

Q773. Nearest greater element?

A: Monotonic stack.

Q774. Window maximum?

A: Monotonic deque.

Q775. Prefix autocomplete?

A: Trie.

Q776. Large integer multiplication?

A: String digit multiplication.

Q777. Longest palindromic substring in $O(n)$?

A: Manacher.

Q778. Pattern matching in $O(n+m)$?

A: KMP or Z algorithm.

Q779. Range update?

A: Difference array for offline/simple cases; lazy segment tree for dynamic range operations.

Q780. Interview golden rule?

A: Match the problem's constraints to the right pattern before writing code.

Coverage Checklist

This rapid-fire bank covers the supplied roadmap's major sections and their listed problem families:

- Fundamentals and complexity analysis
- Mathematics, patterns, recursion
- Arrays, strings, matrices
- Linear search, binary search, search-on-answer, median/kth across sorted arrays
- Sorting, partitioning, Merge Sort, Quick Sort, Cycle Sort
- Bit manipulation and bitwise array/string techniques
- Hashing and frequency/subsequence problems
- Two-pointer techniques
- Sliding-window techniques, including hashing-based windows
- Prefix sums, suffix transformations, difference arrays, and prefix hashing
- Backtracking
- Singly, doubly, circular, and advanced linked lists
- Stack, expressions, monotonic stack, and stack-design problems
- Queue and deque
- Binary trees and advanced traversal/view/path problems
- Binary Search Trees
- Heaps and priority queues
- Graph representation, BFS/DFS, cycles, bipartite graphs
- Topological sorting and dependency problems
- Shortest paths
- Multi-source BFS
- MST and DSU
- Greedy algorithms
- Dynamic programming: 1-D, 2-D, subsequence, grid, string, stock, LIS, partition
- Number theory
- Trie and binary Trie
- String matching: Rabin-Karp, KMP, Z, Manacher
- Segment Tree, Lazy Propagation, Fenwick Tree
- Complexity and pattern-recognition rapid revision

Source basis: the supplied DSA roadmap organizes these topics and problem families across its Fundamentals through Range Query sections. [filecite\turn0file0\L5-L8](#) [filecite\turn1file0\L16-L23](#) [filecite\turn1file0\L25-L42](#) [filecite\turn1file0\L44-L67](#)

29. Fundamentals — Deeper Rapid Fire

Q781. What is the difference between worst-case and average-case complexity?

A: Worst-case measures the maximum cost for any valid input; average-case measures expected cost under an assumed input distribution.

Q782. What is best-case complexity?

A: The minimum running time over valid inputs of a given size.

Q783. Why are constants usually ignored in Big-O?

A: Asymptotic analysis focuses on growth as input size becomes large, so constant factors do not change the growth class.

Q784. What is logarithm base relevance in complexity?

A: For constant bases greater than 1, logarithms differ only by a constant factor, so they have the same asymptotic complexity.

Q785. What is exponential complexity?

A: Complexity such as $O(2^n)$, where increasing n by one can roughly multiply the work by a constant factor.

Q786. What is factorial complexity?

A: $O(n!)$, typically associated with enumerating all permutations.

Q787. What is a brute-force solution?

A: A straightforward solution that systematically explores possibilities without exploiting problem-specific optimization.

Q788. When is brute force useful?

A: For small constraints, as a correctness baseline, or as a way to discover patterns before optimizing.

Q789. What is asymptotic dominance?

A: For large n , the fastest-growing term dominates the running time, such as n^2 dominating n .

Q790. What is a time-space trade-off?

A: Using additional memory to reduce computation time, such as a hash map replacing repeated searches.

Q791. What is preprocessing?

A: Work performed before answering queries, such as sorting or building prefix sums.

Q792. What is output-sensitive complexity?

A: Complexity expressed partly in terms of the amount of output produced, important for enumeration problems.

Q793. What is an invariant in a two-pointer algorithm?

A: A condition describing what the current pointer interval represents and why discarded elements cannot form a better/valid solution.

Q794. What is correctness proof by contradiction?

A: Assume the algorithm's result is incorrect and derive a contradiction with an established property or invariant.

Q795. What is an exchange argument?

A: Show that an optimal solution can be transformed to use the greedy choice without making it worse.

Q796. What is optimal substructure?

A: An optimal solution can be constructed from optimal solutions to relevant smaller subproblems.

30. Arrays — Deeper Rapid Fire

Q797. What is the difference between a subarray and a subsequence?

A: A subarray is contiguous; a subsequence preserves order but may skip elements.

Q798. What is a subset?

A: A collection selected from the original elements where order generally does not matter.

Q799. How many subsequences does an n -element sequence have?

A: 2^n , including the empty subsequence.

Q800. How many non-empty subsequences are there?

A: $2^n - 1$.

Q801. How many subarrays contain a particular index i ?

A: $(i+1)(n-i)$.

Q802. How do you find a duplicate when values are in $1..n$?

A: Depending on constraints, use a set, sign marking, cyclic placement, or Floyd's cycle technique.

Q803. Why can Floyd's algorithm find a duplicate in an array?

A: Treat values as next pointers; the duplicate creates a cycle.

Q804. What is cyclic sort useful for?

A: Arrays whose values map naturally to target indices, especially missing/duplicate number problems.

Q805. How do you find missing numbers from 1..n?

A: Use cyclic placement, marking, XOR, or sum-based methods depending on constraints.

Q806. What is the trapping-rain-water prefix idea?

A: Water at i is $\max(0, \min(\text{leftMax}, \text{rightMax}) - \text{height}[i])$.

Q807. What is the maximum product of two elements pattern?

A: Track the two largest values, or account for the two smallest negatives.

Q808. How do you find the minimum difference between any two array elements?

A: Sort the array and inspect adjacent differences.

Q809. How do you find common elements while preserving duplicates?

A: Use frequency counting or sorted two-pointer matching.

Q810. How do you find the longest consecutive sequence?

A: Use a hash set and start counting only at numbers with no predecessor.

Q811. Why does longest consecutive sequence run in $O(n)$ expected time?

A: Each sequence is expanded only from its smallest element, so values are not repeatedly traversed.

Q812. What is the maximum subarray with one deletion pattern?

A: Maintain DP states for the best sum ending at the current position with and without one deletion.

Q813. What is the maximum alternating subarray pattern?

A: Track whether the current valid sequence expects an increase or decrease.

Q814. How do you find the majority element when it may not exist?

A: Boyer-Moore finds a candidate; verify its frequency afterward.

Q815. What is the advantage of an in-place partition?

A: It reduces auxiliary memory while rearranging elements directly in the input.

Q816. What is stable partition?

A: Partitioning while preserving relative order within each resulting group.

31. Searching — Advanced

Q817. What is binary search on a monotonic predicate?

A: Find the boundary where a boolean condition changes from false to true or true to false.

Q818. What is first true?

A: The smallest index/value for which a monotonic predicate becomes true.

Q819. What is last true?

A: The largest index/value for which a monotonic predicate remains true.

Q820. Why can binary search fail on unsorted data?

A: The ordering property needed to discard half the search space is absent.

Q821. What is exponential search?

A: Expand the search range exponentially until the target may lie inside, then binary search that range.

Q822. When is exponential search useful?

A: For conceptually unbounded sorted sequences or when the target may be near the beginning.

Q823. What is interpolation search?

A: It estimates the likely target position using value distribution; it can be fast on uniformly distributed data but has poor worst-case behavior.

Q824. How do you find the minimum in a rotated sorted array?

A: Compare the middle with the right boundary and determine which sorted half contains the rotation point.

Q825. How do you count rotations in a sorted rotated array?

A: The index of the minimum element gives the rotation count under the standard left-rotation convention.

Q826. What is a bitonic array?

A: An array that increases and then decreases, usually with a single peak.

Q827. How do you find a peak in a bitonic array?

A: Binary search by comparing adjacent values.

Q828. How do you search in a bitonic array?

A: Find the peak, then binary-search the increasing and decreasing halves.

Q829. What is a mountain array?

A: An array that strictly increases to one peak and strictly decreases afterward.

Q830. What is the minimum capacity shipping problem?

A: Binary search shipment capacity and greedily test whether all packages can be shipped within the required days.

Q831. What is painter partition?

A: Binary search the maximum workload assigned to one painter and check whether all boards can be covered within the painter limit.

32. Sorting — Advanced

Q832. What makes a sorting algorithm stable?

A: Equal-key elements retain their original relative order.

Q833. What makes a sorting algorithm adaptive?

A: It can run faster when the input is already or nearly sorted.

Q834. What is comparison sorting?

A: Sorting based on comparisons between elements.

Q835. What lower bound applies to comparison sorting?

A: $\Omega(n \log n)$ comparisons in the general comparison model.

Q836. Why can counting sort beat $n \log n$?

A: It is not comparison-based and exploits a bounded integer key range.

Q837. What is counting sort complexity?

A: $O(n+k)$, where k is the value/key range.

Q838. What is radix sort?

A: Sort numbers digit by digit using a stable intermediate sorting method.

Q839. What is bucket sort?

A: Distribute elements into buckets, sort each bucket, then concatenate them.

Q840. What is the Dutch National Flag invariant?

A: Maintain regions for processed 0s, unknown values, and processed 2s, with 1s handled in the middle.

Q841. Why is Merge Sort useful for linked lists?

A: It does not require random access and can merge lists efficiently with $O(1)$ pointer-based auxiliary space apart from recursion.

Q842. Why is Quick Sort less attractive for linked lists?

A: Partitioning and random access are less natural, although specialized implementations are possible.

Q843. What is quickselect?

A: A partition-based selection algorithm for finding the k th smallest/largest without fully sorting.

Q844. Average complexity of Quickselect?

A: $O(n)$ expected time.

Q845. Worst-case Quickselect complexity?

A: $O(n^2)$ with consistently poor pivots.

Q846. What does median-of-medians provide?

A: A deterministic pivot strategy giving linear worst-case selection complexity.

33. Bit Manipulation — Advanced

Q847. What is a bit mask?

A: An integer whose bits encode boolean states or selections.

Q848. How do you set a bit?

A: $n \mid (1 \ll k)$.

Q849. How do you clear a bit?

A: $n \& \sim(1 \ll k)$.

Q850. How do you toggle a bit?

A: $n \wedge (1 \ll k)$.

Q851. How do you test whether a bit is zero?

A: Check $(n \& (1 \ll k)) == 0$.

Q852. What does left shift generally do?

A: For unsigned/non-overflowing integer contexts, $n \ll k$ multiplies by 2^k .

Q853. What does right shift generally do?

A: It shifts bits toward the least significant side; signed behavior depends on language/type semantics.

Q854. What is subset enumeration using bit masks?

A: Represent each subset by an n -bit number where bit i indicates whether element i is selected.

Q855. How many masks represent subsets of n elements?

A: 2^n .

Q856. How do you enumerate set bits?

A: Repeatedly extract/clear the lowest set bit.

Q857. What is XOR basis?

A: A linear-basis technique over bits used to represent and optimize XOR combinations.

Q858. What is the maximum XOR subset problem?

A: Build a linear XOR basis and greedily improve the result from high bits downward.

Q859. What is bitwise complement?

A: It flips every bit in the represented width; the meaningful width must be specified for bounded binary representations.

Q860. Why should signed shifts be handled carefully?

A: Sign extension and language-specific behavior can affect results.

Q861. What is the bit-count identity for powers of two?

A: A positive power of two has exactly one set bit.

34. Hashing — Advanced

Q862. What is a hash function?

A: A function that maps a key to a hash value used to select a bucket/index.

Q863. What properties should a good hash function have?

A: It should distribute common inputs uniformly, be deterministic, and be efficient to compute.

Q864. What is collision resolution?

A: The mechanism used when multiple keys map to the same location.

Q865. What is open addressing?

A: Store all entries directly in the table and probe alternative positions after collisions.

Q866. What is primary clustering?

A: Linear probing can create long contiguous clusters of occupied slots.

Q867. What is secondary clustering?

A: Quadratic probing can cause keys with the same initial hash to follow related probe sequences.

Q868. What is rehashing?

A: Rebuilding the table with a new capacity and reinserting entries.

Q869. Why is hashing average $O(1)$, not guaranteed $O(1)$?

A: Poor distributions or adversarial collisions can degrade performance.

Q870. What is frequency-map compression?

A: Store only distinct values and their counts instead of every occurrence.

Q871. How do you group anagrams?

A: Use a canonical signature such as sorted characters or a frequency vector as the hash key.

Q872. How do you find the longest substring with exactly K distinct characters?

A: Sliding window plus a frequency map.

Q873. How do you count pairs with a target difference?

A: Use a set/frequency map and look for the complementary value.

Q874. How do you find duplicate rows in a binary matrix?

A: Hash each row or encode it efficiently and count repeated representations.

Q875. What is a rolling hash?

A: A hash that can be updated efficiently when a window shifts.

Q876. What is hash collision risk in string algorithms?

A: Different strings may share a hash; use verification or multiple hashes when correctness requires strong collision resistance.

35. Two Pointers — Advanced

Q877. What is the three-way partition pattern?

A: Maintain boundaries for values smaller than, equal to, and larger than a pivot.

Q878. What is the remove-duplicates-from-sorted-array pattern?

A: Fast pointer scans; slow pointer writes the next unique value.

Q879. How do you remove all occurrences of a value in place?

A: Use a write pointer to overwrite unwanted values.

Q880. What is partition around a pivot?

A: Rearrange elements so those satisfying one relation to the pivot are on one side and the others on the other

side.

Q881. What is the fast/slow pointer pattern beyond linked lists?

A: It can compact arrays, detect cycles in functional graphs, and find certain structural boundaries.

Q882. How do you find a pair with a given difference in a sorted array?

A: Maintain two pointers and adjust based on the current difference.

Q883. What is 3Sum duplicate handling?

A: Skip repeated fixed values and repeated pointer values after recording a valid triplet.

Q884. Why is sorting often acceptable in 3Sum?

A: It enables $O(n^2)$ two-pointer processing and simplifies duplicate elimination.

Q885. What is the 4Sum overflow concern?

A: Adding several integers can overflow; use a sufficiently wide integer type.

Q886. What is partition-label style thinking?

A: Determine boundaries so each component can be finalized without future elements affecting it.

36. Sliding Window — Advanced

Q887. What is a window invariant?

A: A condition that defines what the current window contains and ensures the algorithm's answer remains valid.

Q888. Why do sliding-window algorithms often run in $O(n)$?

A: Each pointer generally moves only forward, so each element enters and leaves a bounded number of times.

Q889. When does sliding window fail for arbitrary signed sums?

A: Negative values can destroy the monotonic relation between expanding a window and its sum.

Q890. What alternative helps with signed subarray sums?

A: Prefix sums with hashing or specialized ordered structures, depending on the requirement.

Q891. What is a monotonic queue?

A: A deque maintaining candidates in monotonic value order for efficient extrema queries.

Q892. What is the shortest subarray with sum at least K when negatives exist?

A: Prefix sums plus a monotonic deque.

Q893. Why is a monotonic deque needed there?

A: It removes dominated prefix sums while preserving candidate indices in increasing order.

Q894. What is at-most-K technique?

A: Count or optimize windows satisfying a condition with at most K violations/distinct items.

Q895. How can exactly-K problems be transformed?

A: Often by subtracting the at-most-K-1 result from the at-most-K result.

Q896. What is a permutation-in-string problem?

A: Determine whether a text contains a substring that is an anagram of a pattern.

Q897. How is permutation-in-string solved?

A: Maintain character frequencies in a fixed-size sliding window.

37. Prefix Sum / Difference — Advanced

Q898. What is a prefix XOR?

A: XOR of all elements from the beginning through a position.

Q899. How do you calculate XOR of range [l,r]?

A: `prefixXor[r] ^ prefixXor[l-1]`.

Q900. What is a prefix minimum?

A: The minimum value encountered from the beginning through each index.

Q901. What is a suffix maximum?

A: The maximum value encountered from each index through the end.

Q902. How do prefix/suffix arrays help trapping rain water?

A: They provide the left and right maximum boundaries for every position.

Q903. What is a 2D prefix sum inclusion-exclusion formula used for?

A: Computing rectangle sums by adding the full prefix and subtracting overlapping top/left regions.

Q904. What is a difference array's main limitation?

A: It is ideal for certain batch/offline range updates but does not by itself provide arbitrary dynamic range queries efficiently.

Q905. What is coordinate compression useful for with prefix/range structures?

A: It maps sparse large coordinates into manageable dense indexes.

Q906. How do you find the number of subarrays with sum divisible by K?

A: Count equal prefix-sum remainders modulo K.

Q907. Why does equal remainder imply divisibility?

A: Their difference is divisible by K.

38. Backtracking — Advanced

Q908. What is the search tree?

A: A conceptual tree where each node represents a partial decision and each edge represents a choice.

Q909. What is branch-and-bound?

A: Prune branches using bounds showing they cannot beat the current best solution.

Q910. What is constraint propagation?

A: Use existing assignments to eliminate impossible choices before deeper recursion.

Q911. How can permutations with duplicates be generated without duplicates?

A: Sort values and skip equal choices at the same recursion depth.

Q912. How can combinations avoid duplicates?

A: Sort candidates and skip equal candidates at the same depth while controlling the next starting index.

Q913. What is subset generation?

A: At every element, choose include or exclude, creating a binary decision tree.

Q914. What is combination sum?

A: Find combinations of candidate values that reach a target under the problem's reuse rules.

Q915. What is N-Queens diagonal encoding?

A: Cells share a main diagonal when $row - col$ is equal and an anti-diagonal when $row + col$ is equal.

Q916. Why can bit masks accelerate N-Queens?

A: Column and diagonal occupancy can be represented as integer bit sets, making conflict checks very fast.

Q917. What is a Hamiltonian path?

A: A path visiting every vertex exactly once.

Q918. What makes Hamiltonian problems difficult?

A: General Hamiltonian path/cycle search is combinatorial and can require exponential backtracking.

Q919. What is memoization in grid backtracking?

A: Cache results for states where the future result depends only on a compact state representation.

39. Linked Lists — Advanced

Q920. Why use a dummy/sentinel node?

A: It simplifies insertion/deletion near the head by eliminating special-case head handling.

Q921. How do you reverse a linked list recursively?

A: Recursively reverse the suffix, then point the next node back to the current node and clear the old forward link.

Q922. How do you detect the cycle entry?

A: After slow/fast pointers meet, move one pointer to the head; advance both one step until they meet again.

Q923. Why does the cycle-entry method work?

A: The distances traveled modulo the cycle length imply that the second meeting occurs at the cycle entrance.

Q924. How do you merge-sort a linked list?

A: Split using slow/fast pointers, recursively sort both halves, then merge.

Q925. What is an odd-even linked list?

A: Rearrange nodes so nodes at odd positions precede nodes at even positions while preserving relative order within each group.

Q926. What is a reorder-list problem?

A: Transform $L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_n$ into $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow \dots$

Q927. Standard reorder-list strategy?

A: Find middle, reverse second half, then merge alternating nodes.

Q928. What is a palindrome linked list's $O(1)$ -space strategy?

A: Reverse the second half and compare it with the first half.

Q929. What is a skip list?

A: A probabilistic layered linked structure supporting expected $O(\log n)$ search, insertion, and deletion.

Q930. What is the advantage of a doubly linked list in LRU?

A: It allows $O(1)$ removal of an arbitrary node when its node reference is known.

Q931. What is the key difficulty in cloning random-pointer lists?

A: Random pointers may reference arbitrary nodes, so old-to-new correspondence must be preserved.

40. Stack — Advanced

Q932. What is a monotonic increasing stack?

A: A stack maintained so values increase from bottom to top, useful for smaller-element queries.

Q933. What is a monotonic decreasing stack?

A: A stack maintained in decreasing value order, useful for greater-element queries.

Q934. Why are indices often stored instead of values?

A: Indices provide positions needed to calculate distances and widths.

Q935. What is the contribution technique for subarray minimums?

A: For each element, determine how many subarrays choose it as their minimum and add its weighted contribution.

Q936. What ambiguity must be handled in contribution methods with duplicates?

A: Equal values need asymmetric strict/non-strict boundary rules to avoid double counting.

Q937. What is the daily temperatures pattern?

A: Monotonic decreasing stack of indices finds the next greater temperature.

Q938. What is asteroid collision?

A: Simulate collisions using a stack, resolving only when a positive asteroid meets a negative asteroid in the relevant direction.

Q939. What is next greater element in a circular array?

A: Process the array twice conceptually using a monotonic stack.

Q940. What is maximal rectangle?

A: Treat each matrix row as a histogram and solve largest rectangle repeatedly.

Q941. What is the largest rectangle histogram boundary?

A: For each bar, nearest smaller elements on both sides determine its maximum width.

41. Queue / Deque — Advanced

Q942. What is BFS level-order processing?

A: Process all nodes currently in the queue before processing the next level.

Q943. How do you compute minimum steps in an unweighted state graph?

A: BFS from the start state.

Q944. What is 0-1 BFS?

A: A shortest-path algorithm for graphs whose edge weights are only 0 or 1, using a deque.

Q945. Why does 0-1 BFS use a deque?

A: Zero-cost transitions go to the front; one-cost transitions go to the back.

Q946. Complexity of 0-1 BFS?

A: $O(V+E)$.

Q947. What is a priority queue?

A: An abstract structure supporting access to the highest- or lowest-priority element.

Q948. Is a priority queue necessarily a heap?

A: No; a heap is a common implementation, but other structures can implement the abstraction.

Q949. What is a circular buffer?

A: Fixed-size storage where indices wrap around, commonly used for queues and streaming data.

Q950. What is a blocking queue conceptually?

A: A queue where producers/consumers may wait when the queue is full/empty.

Q951. What is monotonic queue dominance?

A: An element that is worse than a newer element for every future relevant window can be removed.

42. Trees — Advanced

Q952. What is a full binary tree?

A: Every node has either zero or two children.

Q953. What is a complete binary tree?

A: Every level except possibly the last is full, and the last level is filled left to right.

Q954. What is a perfect binary tree?

A: Every internal node has two children and all leaves are at the same depth.

Q955. What is a skewed binary tree?

A: A tree where nodes predominantly have only one child, producing chain-like height.

Q956. What is the maximum number of nodes at level d ?

A: 2^d for zero-based depth.

Q957. Maximum nodes in a binary tree of height h ?

A: $2^{(h+1)}-1$ when height counts edges from root.

Q958. Minimum height for n nodes?

A: Approximately $\text{ceil}(\log_2(n+1))-1$ under edge-based height.

Q959. What is complete-tree array indexing useful for?

A: Heap representation and certain level-indexed tree algorithms.

Q960. What is threaded binary tree traversal?

A: Use otherwise-null pointers to store traversal successor/predecessor links temporarily or permanently.

Q961. What is an Euler tour of a tree?

A: A traversal representation that records entry/exit events, useful for subtree/range transformations.

Q962. How can subtree queries be reduced to array queries?

A: Flatten the tree with DFS entry/exit times so each subtree becomes a contiguous interval.

Q963. What is Lowest Common Ancestor by binary lifting?

A: Precompute 2^j -th ancestors and jump nodes upward efficiently.

Q964. Complexity of LCA with binary lifting?

A: $O(\log n)$ per query after $O(n \log n)$ preprocessing.

Q965. What is tree DP?

A: Dynamic programming where each node combines results from its child subtrees.

Q966. What is rerooting DP?

A: Compute answers for one root and propagate information so answers for every possible root can be obtained efficiently.

43. BST — Advanced

Q967. Why is a balanced BST valuable?

A: It keeps height near $O(\log n)$, preserving efficient search/insert/delete.

Q968. What are AVL trees?

A: Self-balancing BSTs maintaining a strict height-balance condition.

Q969. What are Red-Black trees?

A: Self-balancing BSTs using node colors and structural rules to bound tree height.

Q970. What is the purpose of BST rotations?

A: Restore balance or restructure the tree while preserving inorder ordering.

Q971. What is an AVL balance factor?

A: Height(left subtree) minus height(right subtree), commonly required to be -1, 0, or 1.

Q972. What is a predecessor/successor shortcut in a BST?

A: The predecessor is the rightmost node of the left subtree; successor is the leftmost node of the right subtree when those subtrees exist.

Q973. How can a BST become degenerate?

A: Inserting already sorted keys into an ordinary unbalanced BST can create a chain.

Q974. What is a range query in a BST?

A: Visit only branches whose values could fall inside the requested interval.

Q975. What is a two-sum BST iterator?

A: One iterator produces values ascending and another descending, allowing a two-pointer-like search without flattening.

Q976. What is BST from preorder?

A: Reconstruct the tree by maintaining valid value bounds while consuming preorder values.

44. Heap — Advanced

Q977. Why is build-heap $O(n)$?

A: Most nodes are near the leaves and require little heapification work; the total geometric sum is linear.

Q978. What is heapify?

A: Restore heap order from a node downward or build the heap from an array.

Q979. What is decrease-key?

A: Reduce a key in a min-priority structure and move it upward as necessary.

Q980. What is increase-key?

A: Increase a key in a max-oriented structure and restore heap order upward as appropriate.

Q981. What is a d-ary heap?

A: A heap where each node has d children, trading shallower height against more child comparisons.

Q982. When is a heap preferable to sorting for top K ?

A: When K is much smaller than n or when data arrives as a stream.

Q983. How do you maintain the smallest K values in a stream?

A: Use a max-heap of size K .

Q984. How do you maintain the largest K values in a stream?

A: Use a min-heap of size K .

Q985. What is a min-max heap?

A: A specialized heap supporting efficient access to both minimum and maximum elements.

Q986. Why are two heaps useful for median?

A: They partition the data into lower and upper halves while exposing each half's boundary.

45. Graphs — Advanced

Q987. What is graph degree?

A: Number of incident edges; in directed graphs, distinguish indegree and outdegree.

Q988. What is a simple graph?

A: A graph without self-loops or parallel edges under the standard definition.

Q989. What is a multigraph?

A: A graph that may contain multiple edges between the same pair of vertices.

Q990. What is a DAG?

A: Directed Acyclic Graph.

Q991. What is a strongly connected component?

A: A maximal set of vertices in a directed graph where every vertex can reach every other.

Q992. Which algorithms find SCCs?

A: Kosaraju and Tarjan are standard algorithms.

Q993. What is a bridge?

A: An edge whose removal increases the number of connected components.

Q994. What is an articulation point?

A: A vertex whose removal increases the number of connected components.

Q995. Which technique finds bridges/articulation points efficiently?

A: DFS with discovery times and low-link values.

Q996. What is a strongly connected DAG condensation?

A: Contract each SCC into one vertex; the resulting component graph is a DAG.

Q997. What is a bipartite graph's odd-cycle property?

A: A graph is bipartite if and only if it contains no odd cycle.

Q998. What is graph coloring?

A: Assign colors to vertices so adjacent vertices satisfy the required color constraint.

Q999. What is a complete graph?

A: Every pair of distinct vertices has an edge in the undirected case.

Q1000. What is a tree as a graph?

A: A connected undirected graph with no cycles.

Q1001. How many edges does an undirected tree with V vertices have?

A: V-1.

Q1002. What is a forest?

A: An undirected graph containing no cycles; each component is a tree.

Q1003. What is a DAG useful for?

A: Dependency ordering, scheduling, and DP over directed dependencies.

Q1004. What is shortest path in a DAG?

A: Process vertices in topological order and relax outgoing edges; negative edges are allowed because cycles are absent.

Q1005. What is A* search?

A: A shortest-path/search algorithm using path cost plus a heuristic estimate to prioritize promising states.

Q1006. What makes an A* heuristic admissible?

A: It never overestimates the true remaining cost.

Q1007. What is transitive closure?

A: Determine whether a path exists between every pair of vertices.

Q1008. What is Warshall's algorithm?

A: A dynamic-programming algorithm for transitive closure; Floyd-Warshall generalizes the idea to weighted shortest paths.

Q1009. What is a negative cycle?

A: A cycle whose total edge weight is negative.

Q1010. Why is a reachable negative cycle problematic for shortest paths?

A: Paths can repeatedly traverse it to reduce cost without bound.

Q1011. What is edge relaxation?

A: Attempt to improve $\text{dist}[v]$ using $\text{dist}[u] + \text{weight}(u, v)$.

46. MST / DSU — Advanced

Q1012. What is the cut property of MSTs?

A: A minimum-weight edge crossing a cut is safe for some MST, with uniqueness qualifications when weights tie.

Q1013. What is the cycle property?

A: A maximum-weight edge on a cycle can be excluded from some MST, subject to tie considerations.

Q1014. Prim vs Kruskal?

A: Prim grows one tree from a starting vertex; Kruskal globally processes edges and merges components.

Q1015. When is Kruskal attractive?

A: Especially for sparse graphs where edge sorting and DSU are convenient.

Q1016. What does DSU find?

A: The representative/root of the set containing an element.

Q1017. What is union by size?

A: Attach the smaller DSU tree under the larger tree's root.

Q1018. What is path compression?

A: During find, make traversed nodes point directly to the set representative.

Q1019. Can DSU find directed cycles?

A: Standard DSU is naturally suited to undirected connectivity; directed cycle detection generally needs different techniques.

Q1020. What happens to DSU component count after a successful union?

A: It decreases by one.

Q1021. What is offline connectivity?

A: Process connectivity operations in a chosen order, often allowing DSU to answer incremental unions efficiently.

47. Greedy — Advanced

Q1022. Why does greedy sometimes fail?

A: A locally optimal choice may block a better global combination when the problem lacks the required greedy-

choice property.

Q1023. What is the canonical greedy interval rule?

A: For maximizing compatible activities, choose the earliest finishing interval.

Q1024. What is the canonical greedy fractional-knapsack rule?

A: Choose highest value-to-weight ratio first.

Q1025. What is the canonical Huffman rule?

A: Repeatedly merge the two least frequent symbols.

Q1026. What is an exchange proof?

A: Replace the first choice in an optimal solution with the greedy choice and show feasibility and objective value are preserved or improved.

Q1027. What is a matroid intuition for greedy?

A: In matroid-like systems, local maximum-weight independent choices can produce a global optimum.

Q1028. Why does Dijkstra use greedy selection?

A: With nonnegative edges, the smallest tentative distance cannot later be improved through an unprocessed vertex.

Q1029. Is Dijkstra a greedy or DP algorithm?

A: It is fundamentally greedy, though it uses relaxation and maintains accumulated state.

Q1030. What is minimum number of coins with arbitrary denominations?

A: Greedy is not universally correct; DP is the safe general approach.

Q1031. When is greedy coin change guaranteed?

A: For certain canonical denomination systems, not for arbitrary denominations.

48. Dynamic Programming — Advanced

Q1032. How do you recognize DP?

A: Look for repeated subproblems, choices, optimization/counting, and a compact state describing the remaining problem.

Q1033. What is a DP transition?

A: The formula that derives a state from previously computed states.

Q1034. What is a base state?

A: The smallest/initial state whose answer is known directly.

Q1035. What is state-space complexity?

A: Number of distinct DP states multiplied by the work needed to transition from each state.

Q1036. What is dimension reduction in DP?

A: Store only the previous states needed for transitions rather than the entire table.

Q1037. Why can 2D DP become 1D?

A: If each row depends only on the previous row/current row in a controlled direction.

Q1038. What is bitmask DP?

A: Use a bit mask to represent a subset/state, often for small n .

Q1039. Typical complexity of subset bitmask DP?

A: Often $O(2^n \cdot \text{poly}(n))$, depending on transitions.

Q1040. What is interval DP?

A: DP over contiguous intervals, common in matrix-chain multiplication and partition problems.

Q1041. What is tree DP?

A: Compute states for each tree node based on child states.

Q1042. What is digit DP?

A: Count or optimize numbers satisfying digit constraints using states such as position, tightness, and accumulated properties.

Q1043. What is the tight flag in digit DP?

A: It indicates whether the constructed prefix is equal to the bound's prefix, restricting the next digit.

Q1044. What is LCS reconstruction?

A: Backtrack through the DP table to recover one longest common subsequence.

Q1045. What is Hirschberg's algorithm?

A: An LCS algorithm that reduces auxiliary space to linear while retaining $O(mn)$ time.

Q1046. What is LIS tails array actually storing?

A: For each subsequence length, the smallest possible tail value, not necessarily an actual subsequence.

Q1047. Why can LIS tails use binary search?

A: Tails are maintained in sorted order, allowing replacement position lookup in $O(\log n)$.

Q1048. What is weighted interval scheduling?

A: Select non-overlapping intervals maximizing total weight; sort by finish time and use DP with predecessor lookup.

Q1049. What is longest palindromic subsequence?

A: Longest sequence that reads the same forward/backward, not necessarily contiguous.

Q1050. Relationship between LPS and LCS?

A: LPS length can be computed as $\text{LCS}(\text{string}, \text{reverse}(\text{string}))$, with caveats about reconstruction/interpretation.

Q1051. What is minimum edit distance optimization?

A: Choose insert, delete, or replace transitions minimizing total operations.

49. Number Theory — Advanced

Q1052. What is GCD?

A: Greatest common divisor of two integers.

Q1053. What is LCM?

A: Least common multiple; for nonzero a, b , $\text{lcm}(a, b) = |a / \text{gcd}(a, b) * b|$ using overflow-safe ordering.

Q1054. What is modular inverse?

A: A value x such that $a * x \equiv 1 \pmod{m}$, when the inverse exists.

Q1055. When does a modular inverse exist?

A: Exactly when $\text{gcd}(a, m) = 1$.

Q1056. What is Fermat's little theorem useful for?

A: For prime modulus p and a not divisible by p , $a^{(p-1)} \equiv 1 \pmod{p}$, enabling modular inverses.

Q1057. What is extended Euclidean algorithm?

A: It computes $\gcd(a,b)$ and coefficients x,y satisfying $ax+by=\gcd(a,b)$.

Q1058. What is Chinese Remainder Theorem?

A: It determines a unique residue modulo the product for compatible congruences with pairwise coprime moduli.

Q1059. What is fast doubling Fibonacci?

A: Compute Fibonacci numbers using identities that halve the index recursively, giving $O(\log n)$ time.

Q1060. What is a perfect number?

A: A positive integer equal to the sum of its proper positive divisors.

Q1061. What is a highly composite idea?

A: A number having more divisors than every smaller positive integer.

Q1062. What is a prime factorization?

A: Representation of an integer as a product of prime powers.

Q1063. Why is factorization important in Euler Totient?

A: $\varphi(n)$ is computed from the distinct prime factors of n .

Q1064. What is modular multiplication overflow concern?

A: Multiplying two values before taking modulo may overflow; use wider types or safe modular multiplication when necessary.

Q1065. What is sieve preprocessing useful for?

A: Fast primality checks and smallest-prime-factor-based factorization across many queries.

Q1066. What is SPF?

A: Smallest Prime Factor for each integer, often precomputed with a sieve.

50. Trie — Advanced

Q1067. What is prefix uniqueness?

A: A prefix uniquely identifies a word when no other stored word shares that prefix, subject to the dictionary.

Q1068. What is Trie memory complexity?

A: $O(\text{total characters})$ nodes, multiplied by per-node child representation overhead.

Q1069. Array children vs hash-map children in a Trie?

A: Arrays provide faster predictable access for small fixed alphabets but use more memory; maps save memory for sparse branching.

Q1070. How do you delete a word from a Trie?

A: Clear its end-of-word marker and optionally remove nodes no longer needed by other words.

Q1071. How do you count words with a prefix?

A: Store subtree word counts at Trie nodes or traverse the prefix and aggregate descendants.

Q1072. How do you implement autocomplete ranking?

A: Store frequency/ranking metadata at nodes or perform a ranked traversal beneath the prefix.

Q1073. What is a compressed Trie?

A: A Trie that compresses chains of single-child nodes into multi-character edges.

Q1074. What is a radix tree?

A: A compressed prefix tree where edges can represent strings rather than single characters.

Q1075. Why is a binary Trie useful for XOR?

A: It chooses bits greedily from most significant to least significant.

Q1076. What is XOR pair counting?

A: Count pairs satisfying an XOR condition using a binary Trie or bitwise digit DP depending on the constraint.

51. String Matching — Advanced

Q1077. What is prefix function?

A: For each position, it stores the length of the longest proper prefix that is also a suffix.

Q1078. Why does KMP avoid repeated comparisons?

A: The prefix function tells how far the pattern can shift while preserving matched prefix information.

Q1079. What is the Z-box?

A: The current interval where the substring matches the prefix in the Z algorithm.

Q1080. Why does Z algorithm run in linear time?

A: The right boundary of the maintained matching interval only moves forward overall.

Q1081. What is double hashing?

A: Use two independent hash functions to greatly reduce collision probability.

Q1082. What is rolling-hash window update?

A: Remove the outgoing character's weighted contribution, shift the hash, and add the incoming character.

Q1083. Why verify Rabin-Karp hash matches?

A: Hash equality does not mathematically guarantee string equality because collisions are possible.

Q1084. What is a palindrome radius?

A: The distance/length parameter describing how far a palindrome extends around a center.

Q1085. Why does Manacher transform the string?

A: A separator-based transformation lets odd- and even-length palindromes be handled uniformly.

Q1086. What is the main advantage of KMP over naive matching?

A: It guarantees linear-time matching without backtracking in the text.

52. Range Query — Advanced

Q1087. What is a segment tree node's merge operation?

A: A function combining the aggregate information of its two child segments.

Q1088. What property makes some segment-tree aggregates easy to merge?

A: Associativity of the underlying operation.

Q1089. Is associativity strictly required for every segment tree?

A: It is fundamental for standard aggregate decomposition, though custom node state can support richer operations.

Q1090. What is a lazy tag?

A: Deferred update information stored at a segment node.

Q1091. What is lazy propagation invariant?

A: A node's stored aggregate reflects its pending update, while descendants may still need that update applied.

Q1092. What is range assignment vs range addition?

A: Assignment replaces values and must override older pending updates appropriately; addition accumulates with existing updates.

Q1093. Why is lazy assignment more complex than addition?

A: A newer assignment invalidates previous values/updates in the affected range and requires careful tag composition.

Q1094. What is coordinate compression with segment trees?

A: Convert sparse coordinates into ordered indices so a tree can operate on a manageable domain.

Q1095. What is a Fenwick tree's lowbit?

A: i & $-i$, representing the size of the range aggregated by index i in the standard one-based implementation.

Q1096. Why does Fenwick update jump by lowbit?

A: Each jump moves to the next tree node whose represented range includes the updated position.

Q1097. Why does Fenwick prefix query subtract lowbit?

A: It moves to the parent prefix block after accumulating the current block.

Q1098. Can Fenwick Tree directly support range minimum with arbitrary updates?

A: Standard Fenwick structure is naturally suited to invertible prefix aggregates such as sums; arbitrary range minimum updates need different techniques.

Q1099. What is a sparse table?

A: A static range-query structure precomputing answers for power-of-two intervals.

Q1100. Sparse table query complexity for idempotent operations?

A: $O(1)$, after $O(n \log n)$ preprocessing, for operations such as minimum/maximum/GCD under standard sparse-table assumptions.

Q1101. What is the major sparse-table limitation?

A: It is primarily for static data; updates are expensive compared with dynamic trees.

53. Mixed Interview Traps

Q1102. Can binary search be used on a linked list efficiently?

A: Not usually; finding the middle is $O(n)$, destroying the usual $O(\log n)$ advantage.

Q1103. Can BFS use a stack?

A: A standard BFS requires queue-like FIFO behavior; using a stack changes it into DFS-like behavior.

Q1104. Can DFS find shortest paths in an unweighted graph?

A: It can sometimes find a path, but standard DFS does not guarantee the shortest path.

Q1105. Can Dijkstra handle zero-weight edges?

A: Yes; zero weights are nonnegative.

Q1106. Can Dijkstra handle negative cycles?

A: No.

Q1107. Can Bellman-Ford handle negative cycles?

A: It can detect reachable negative cycles but cannot provide finite shortest distances for vertices affected by them.

Q1108. Does every connected graph have an MST?

A: Every connected weighted undirected graph has at least one MST.

Q1109. Does every graph have an MST?

A: No; disconnected graphs have a minimum spanning forest instead.

Q1110. Does every DAG have a topological order?

A: Yes.

Q1111. Can a topological ordering be unique?

A: Yes, if at every step there is exactly one valid next vertex under the graph's constraints.

Q1112. Is every binary tree a BST?

A: No; BSTs impose an ordering property.

Q1113. Is every BST balanced?

A: No.

Q1114. Is every complete binary tree a heap?

A: No; it must also satisfy heap order.

Q1115. Is a heap sorted?

A: No; only parent-child ordering is guaranteed.

Q1116. Is a hash table ordered?

A: Not inherently; ordering depends on the particular implementation.

Q1117. Is a Trie good for arbitrary numeric range queries?

A: Not generally; its strength is prefix/bitwise structure.

Q1118. Is DP always faster than greedy?

A: No; when greedy is correct, it is often simpler and faster.

Q1119. Is recursion always slower than iteration?

A: Not necessarily; recursion may have stack overhead, but asymptotic complexity can be identical.

Q1120. Is $O(1)$ always better than $O(\log n)$?

A: Asymptotically yes for sufficiently large n , but constants, memory, cache behavior, and implementation constraints also matter.

Q1121. Does $O(n)$ mean exactly n operations?

A: No; it means growth is bounded by a constant multiple of n asymptotically.

54. Final Rapid-Fire Pattern Drill

Q1122. "Find the first/last occurrence" in sorted data?

A: Binary search boundary.

Q1123. "Minimum possible maximum"?

A: Often binary search on answer.

Q1124. "Maximum possible minimum"?

A: Often binary search on answer.

Q1125. "K largest/smallest"?

A: Heap, quickselect, or sorting depending on constraints.

Q1126. "Nearest greater/smaller"?

A: Monotonic stack.

Q1127. "Every window of size K"?

A: Sliding window.

Q1128. "Longest substring satisfying a constraint"?

A: Sliding window is a prime candidate.

Q1129. "Count subarrays with sum K"?

A: Prefix sum + hash map.

Q1130. "Range update many times, final array only"?

A: Difference array.

Q1131. "Dynamic range query + point update"?

A: Fenwick Tree or Segment Tree.

Q1132. "Dynamic range update + range query"?

A: Lazy Segment Tree or a specialized range structure.

Q1133. "Shortest number of moves, all moves equal cost"?

A: BFS.

Q1134. "Shortest path with 0/1 edge weights"?

A: 0-1 BFS.

Q1135. "Shortest path with nonnegative weighted edges"?

A: Dijkstra.

Q1136. "Negative edges without negative cycle"?

A: Bellman-Ford or DAG shortest path when the graph is a DAG.

Q1137. "All-pairs shortest path, small/medium dense graph"?

A: Floyd-Warshall is a classic choice.

Q1138. "Dependency prerequisites"?

A: Topological sorting.

Q1139. "Connect components with minimum total edge cost"?

A: MST.

Q1140. "Repeated union/connectivity queries"?

A: DSU.

Q1141. "Prefix dictionary/autocomplete"?

A: Trie.

Q1142. "Maximum XOR"?

A: Binary Trie or XOR basis depending on the problem.

Q1143. "Generate all possibilities"?

A: Backtracking or bitmask enumeration.

Q1144. "Repeated choices + overlapping states"?

A: DP.

Q1145. "Intervals + maximize count"?

A: Greedy earliest finish.

Q1146. "Fractional selection by value/weight"?

A: Greedy ratio.

Q1147. "Two halves around a median"?

A: Two heaps.

Q1148. "Merge K sorted streams"?

A: Min-heap.

Q1149. "Linked-list cycle"?

A: Floyd slow/fast pointers.

Q1150. "Reverse linked list"?

A: Pointer reversal.

Q1151. "Largest histogram rectangle"?

A: Monotonic stack.

Q1152. "Maximum rectangle of 1s"?

A: Histogram + monotonic stack.

Q1153. "Palindrome in $O(1)$ extra linked-list space"?

A: Reverse the second half and compare.

Q1154. "LCS / Edit Distance"?

A: 2D string DP.

Q1155. "LIS"?

A: $O(n^2)$ DP or $O(n \log n)$ tails method.

Q1156. "Matrix chain / cut interval"?

A: Interval/partition DP.

Q1157. "N-Queens / Sudoku / Word Search"?

A: Backtracking with pruning.

Q1158. "Pattern matching linear time"?

A: KMP or Z algorithm.

Q1159. "Longest palindrome in linear time"?

A: Manacher.

Q1160. "Prime preprocessing up to N"?

A: Sieve of Eratosthenes.

Q1161. "Prime queries over a huge interval"?

A: Segmented sieve.

Q1162. "Fast $a^b \bmod m$ "?

A: Binary/modular exponentiation.

Q1163. "Count coprimes with n"?

A: Euler Totient.

Q1164. "Minimum spanning tree"?

A: Prim or Kruskal.

Q1165. "Shortest range containing one value from every sorted list"?

A: Min-heap + current maximum.

Q1166. "Skyline"?

A: Sweep line + heap/multiset.

Q1167. "No adjacent equal characters"?

A: Greedy max-heap.

Q1168. "Stable matching"?

A: Gale-Shapley.

Q1169. "Minimum cash flow"?

A: Greedy settlement of net balances.

Q1170. "Maximum sum rectangle"?

A: Compress dimensions + Kadane.

Q1171. "Equal 0s and 1s longest subarray"?

A: Convert 0→-1 and use prefix-sum repetition.

Q1172. "Subarray XOR target"?

A: Prefix XOR + frequency map.

Q1173. "Subarray sum divisible by K"?

A: Prefix remainder + frequency/earliest index.

Q1174. "Exactly K distinct"?

A: At-most-K difference.

Q1175. "Minimum window containing pattern"?

A: Frequency map + sliding window.

Q1176. "Search suggestions"?

A: Trie, often with sorted children/results.

Q1177. "Range minimum static array"?

A: Sparse table is an option; segment tree also works.

Q1178. "Range sum + point update"?

A: Fenwick Tree is often simplest.

Q1179. "Complex range update/query"?

A: Segment Tree with lazy propagation.

Q1180. "Tree subtree converted to interval"?

A: DFS Euler/entry-time flattening.

Q1181. "Fast repeated LCA queries"?

A: Binary lifting or another LCA preprocessing method.

Q1182. What is amortized analysis?

A: It analyzes the average cost per operation over a sequence of operations, even when some individual operations are expensive.

Q1183. How is amortized analysis different from average-case analysis?

A: Amortized analysis does not require a probability distribution over inputs; it bounds the total cost of a sequence of operations.

Q1184. What is the accounting method in amortized analysis?

A: Assign each operation an amortized charge, storing excess credit on data structures to pay for future expensive operations.

Q1185. What is the potential method?

A: Define a potential function representing stored work; amortized cost equals actual cost plus the increase in potential.

Q1186. What is a loop invariant?

A: A property that is true before and after every loop iteration and helps prove algorithm correctness.

Q1187. What are precondition and postcondition?

A: A precondition must hold before an algorithm starts; a postcondition must hold after it completes.

Q1188. What is an off-by-one error?

A: An indexing or boundary error caused by using a range that is one position too large or too small.

Q1189. Why are half-open intervals useful?

A: An interval $[l, r)$ has length $r-l$ and composes naturally with adjacent ranges, reducing boundary errors.

Q1190. What is coordinate compression?

A: Mapping large or sparse values to a compact ordered range of indices while preserving relative order.

Q1191. When is coordinate compression useful?

A: When values are huge or sparse but operations depend mainly on ordering or equality, such as Fenwick-tree processing.

Q1192. What is an in-place algorithm?

A: An algorithm that uses $O(1)$ or limited auxiliary memory beyond the input, subject to the stated model.

Q1193. What is a stable algorithm?

A: For equal keys, it preserves their original relative order.

Q1194. Why can stability matter in sorting?

A: Stable sorting allows multiple sorting passes by different keys without destroying earlier ordering.

Q1195. What is an adaptive algorithm?

A: An algorithm whose running time improves when the input already has useful structure, such as nearly sorted data.

Q1196. What is the difference between contiguous and non-contiguous memory?

A: Contiguous elements occupy adjacent addresses; non-contiguous structures store elements separately and connect them through references or indices.

Q1197. How do you find the missing number from $0..n$ in $O(n)$ time and $O(1)$ space?

A: Use XOR: XOR all indices $0..n$ with all array values; equal values cancel and the missing value remains.

Q1198. How do you find two numbers occurring odd numbers of times?

A: XOR all values to get x^y , isolate a set bit, partition values by that bit, then XOR each group separately.

Q1199. How do you move all zeroes to the end while preserving order?

A: Use a write pointer for non-zero values, then fill the remaining positions with zeroes.

Q1200. How do you remove duplicates from a sorted array in-place?

A: Maintain a write pointer; copy each value only when it differs from the last retained value.

Q1201. How do you find the majority element?

A: Use Boyer-Moore voting when a strict majority is guaranteed; otherwise verify the candidate with a second pass.

Q1202. What is the difference between subarray and subsequence?

A: A subarray is contiguous; a subsequence preserves order but may skip elements.

Q1203. How do you find the maximum product subarray?

A: Track both maximum and minimum product ending at each position because multiplying by a negative can

swap their roles.

Q1204. Why is minimum tracking needed for maximum product subarray?

A: A large negative product can become the largest positive product when multiplied by another negative.

Q1205. How do you rotate an array right by k positions in $O(1)$ extra space?

A: Normalize k modulo n, reverse the whole array, then reverse the first k and remaining n-k elements.

Q1206. How do you find an equilibrium index?

A: Compute total sum, maintain a left sum, and check whether left sum equals total minus left sum minus current element.

Q1207. What is the lower_bound operation?

A: It returns the first position whose value is greater than or equal to the target in a sorted range.

Q1208. What is the upper_bound operation?

A: It returns the first position whose value is strictly greater than the target.

Q1209. How do you count occurrences of x in a sorted array?

A: Use $\text{upper_bound}(x) - \text{lower_bound}(x)$.

Q1210. How do you find the first occurrence using binary search?

A: When $\text{arr}[\text{mid}] \geq \text{target}$, record mid if equal and continue searching left.

Q1211. How do you find the last occurrence using binary search?

A: When $\text{arr}[\text{mid}] \leq \text{target}$, record mid if equal and continue searching right.

Q1212. What is binary search on the answer?

A: Binary search over a monotonic feasible answer space instead of directly searching an array.

Q1213. What makes answer-space binary search possible?

A: The feasibility predicate must be monotonic: once an answer becomes feasible, all larger answers, or all smaller answers, must remain feasible.

Q1214. What is exponential search?

A: Check positions 1, 2, 4, 8, ... to locate a range containing the target, then perform binary search inside it.

Q1215. When is interpolation search potentially faster than binary search?

A: On uniformly distributed sorted numeric data, it can estimate the target position and achieve good average performance.

Q1216. Why is binary search often written with $l \leq r$?

A: It represents an inclusive search interval and ensures the final candidate position is examined.

Q1217. Why is quicksort's worst case $O(n^2)$?

A: Highly unbalanced partitions can reduce the problem to sizes $n-1$ and 0 repeatedly.

Q1218. How does randomized pivoting help quicksort?

A: It makes consistently bad pivot choices unlikely for typical adversarial input distributions.

Q1219. Why can quicksort be fast despite $O(n^2)$ worst case?

A: Its partitioning is in-place, cache-friendly, and has low constant factors.

Q1220. What is introsort?

A: A hybrid that starts with quicksort, switches to heapsort when recursion becomes too deep, and often uses insertion sort for small ranges.

Q1221. Why is insertion sort useful for small partitions?

A: Its overhead is tiny and it performs very well on short or nearly sorted sequences.

Q1222. What is counting sort's key limitation?

A: Its complexity depends on the numeric range, which can make it impractical when the range is huge.

Q1223. When is radix sort useful?

A: For fixed-width integers or strings when digits/characters can be processed with a stable linear-time pass.

Q1224. Why must radix-sort passes generally be stable?

A: Stability preserves the ordering established by less significant digits while sorting by more significant digits.

Q1225. What is the decision-tree lower bound for comparison sorting?

A: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Q1226. Can comparison sorting have $O(n)$ worst-case time for arbitrary values?

A: Not under the standard comparison model; additional assumptions about the keys are required.

Q1227. How do you test whether the i-th bit is set?

A: Use $(x \gg i) \& 1$, or equivalently $x \& (1 \ll i)$ when the shift is safe for the type.

Q1228. How do you set a bit?

A: Use $x |= (1 \ll i)$.

Q1229. How do you clear a bit?

A: Use $x \&= \sim(1 \ll i)$.

Q1230. How do you toggle a bit?

A: Use $x \wedge= (1 \ll i)$.

Q1231. How do you remove the lowest set bit?

A: Use $x \&= (x - 1)$.

Q1232. How do you isolate the lowest set bit?

A: Use $x \& -x$ for a signed two's-complement integer representation, or use an appropriate unsigned formulation.

Q1233. How do you check whether a number is a power of two?

A: For a positive integer x , $x \& (x-1)$ must equal zero.

Q1234. How do you count set bits efficiently?

A: Use a hardware/popcount operation when available; otherwise repeatedly clear the lowest set bit.

Q1235. What is XOR's self-inverse property?

A: $a \wedge a = 0$, so equal values cancel when XORed.

Q1236. What is XOR's identity element?

A: 0, because $a \wedge 0 = a$.

Q1237. What is a collision in hashing?

A: Two distinct keys map to the same hash-table bucket.

Q1238. How does separate chaining handle collisions?

A: Each bucket stores a collection of entries, commonly a linked structure or another container.

Q1239. How does open addressing handle collisions?

A: It searches for another slot according to a probing strategy.

Q1240. What is linear probing?

A: On collision, inspect consecutive table positions until an available slot is found.

Q1241. What is quadratic probing?

A: Probe positions using increasing quadratic offsets to reduce primary clustering.

Q1242. What is double hashing?

A: Use a second hash function to determine the probe step, generally reducing clustering.

Q1243. What is load factor in a hash table?

A: The ratio of stored elements to table capacity.

Q1244. Why does high load factor hurt hash-table performance?

A: More collisions occur, increasing the expected number of probes or chain length.

Q1245. What is rehashing?

A: Allocating a larger table and reinserting existing entries using the new capacity/hash mapping.

Q1246. Why can `unordered_map` performance degrade to $O(n)$?

A: Poor hashing or adversarial collisions can make lookup, insertion, or deletion linear in the number of elements.

Q1247. What is the three-way partition technique?

A: Partition values into three regions, commonly less than, equal to, and greater than a pivot.

Q1248. Which problem is naturally solved by Dutch National Flag partitioning?

A: Sorting an array containing three distinct values in one pass and $O(1)$ extra space.

Q1249. How can two sorted arrays be merged?

A: Use two pointers, repeatedly taking the smaller current element.

Q1250. How can you find the intersection of two sorted arrays?

A: Use two pointers; advance the pointer with the smaller value and record matches.

Q1251. What is a common two-pointer pattern for pair sums?

A: Keep one pointer at each end of a sorted array and move inward based on whether the sum is too small or too large.

Q1252. Why does sorting enable many two-pointer solutions?

A: Ordering tells you which pointer movement can safely eliminate impossible pairs.

Q1253. Can two pointers always be used on an unsorted array for two-sum?

A: No; sorting or hashing is typically required unless the input has additional structure.

Q1254. What is fast-slow pointer technique?

A: Two pointers move at different speeds, commonly used for cycle detection and finding a middle node.

Q1255. How does Floyd's cycle detection work?

A: Move slow by one and fast by two; if a cycle exists, they eventually meet.

Q1256. How can you find the cycle entry after a meeting?

A: Reset one pointer to the head and move both one step at a time; their next meeting is the cycle entry.

Q1257. What is a fixed-size sliding window?

A: A window with a constant number of elements that moves across the sequence.

Q1258. What is a variable-size sliding window?

A: A window whose boundaries expand and contract to maintain a condition.

Q1259. When does sliding window usually work?

A: When the condition can be maintained incrementally as elements enter and leave the window.

Q1260. Why is sliding window often $O(n)$?

A: Each element usually enters and leaves the window at most once.

Q1261. How do you find the longest substring without repeating characters?

A: Maintain a left boundary and the last seen position of each character; move left past duplicates.

Q1262. How do you find the minimum window containing required characters?

A: Expand until all requirements are satisfied, then contract from the left while maintaining validity.

Q1263. What is a monotonic queue used for?

A: Maintaining window extrema efficiently, such as maximum or minimum over every fixed-size window.

Q1264. What is the complexity of sliding-window maximum with a deque?

A: $O(n)$, because each element is inserted and removed from the deque at most once.

Q1265. Why can a normal queue fail for sliding-window maximum?

A: It does not efficiently discard elements that can never become the maximum.

Q1266. What does a monotonic decreasing deque store?

A: Candidates in decreasing value order, with the front representing the current maximum.

Q1267. What is a prefix XOR?

A: $\text{prefix}[i]$ is the XOR of elements from the beginning through position i , enabling range XOR queries by XORing prefixes.

Q1268. How do you answer a range sum query using prefix sums?

A: For $[l, r]$, use $\text{prefix}[r] - \text{prefix}[l-1]$, with an appropriate zero-prefix convention.

Q1269. What is a 2D prefix sum?

A: A matrix of cumulative sums that supports rectangle-sum queries in $O(1)$ after $O(\text{rows} \times \text{cols})$ preprocessing.

Q1270. What is a difference array?

A: An auxiliary array where range updates are represented by endpoint changes and recovered using a prefix sum.

Q1271. Why are difference arrays efficient for range additions?

A: Each range update becomes $O(1)$, followed by one $O(n)$ reconstruction pass.

Q1272. What is a sparse prefix structure?

A: A prefix-like representation designed for data with large coordinate spaces or sparse updates, often combined with compression.

Q1273. Can prefix sums handle point updates efficiently?

A: Not generally; a Fenwick tree or segment tree is more suitable when updates and queries are both dynamic.

Q1274. What is a prefix minimum?

A: An array where each position stores the minimum value seen up to that position.

Q1275. Can prefix sums directly solve arbitrary range minimum queries?

A: No; minimum is not invertible like addition, so subtraction of prefixes does not work.

Q1276. What is prefix-function intuition in strings?

A: It records the longest proper prefix that is also a suffix for each prefix of the string.

Q1277. What is pruning in backtracking?

A: Eliminating a partial candidate as soon as it cannot lead to a valid solution.

Q1278. Why is pruning important?

A: It can dramatically reduce the exponential search space.

Q1279. What is branch and bound?

A: A search technique that uses bounds on the best possible completion to eliminate branches that cannot improve the current best.

Q1280. How do you avoid duplicate subsets?

A: For sorted input, skip equal values at the same recursion depth when duplicates would produce identical choices.

Q1281. How do you generate all permutations with duplicates?

A: Sort the input and skip an unused equal value when the previous equal value has not been used at the current recursion level.

Q1282. What is the state in backtracking?

A: The choices currently made, such as selected elements, position, or board configuration.

Q1283. What is the choice-explore-unchoose pattern?

A: Make a choice, recursively explore it, then undo the choice before trying the next alternative.

Q1284. How can N-Queens be optimized?

A: Track occupied columns and diagonals using boolean arrays or bitmasks.

Q1285. What are the two diagonal identifiers in N-Queens?

A: $r-c$ identifies one diagonal family; $r+c$ identifies the other.

Q1286. Why is sorting often useful before backtracking?

A: It can make duplicate elimination and pruning conditions easier.

Q1287. How do you reverse a linked list iteratively?

A: Maintain prev, curr, and next pointers; reverse curr->next, then advance prev and curr.

Q1288. How do you reverse a linked list recursively?

A: Recursively reverse the suffix, then point the next node back to the current node and clear the current next pointer.

Q1289. How do you find the kth node from the end?

A: Move a fast pointer k nodes ahead, then move slow and fast together until fast reaches the end.

Q1290. How do you merge k sorted linked lists?

A: Use a min-heap containing the current node from each list, repeatedly extracting the smallest.

Q1291. What is the complexity of merging k sorted lists with a heap?

A: $O(N \log k)$, where N is the total number of nodes.

Q1292. How do you detect whether two linked lists intersect?

A: Use two pointers that switch to the other list at the end; if they meet, that node is the intersection.

Q1293. Why does pointer switching work for intersection?

A: Both pointers traverse the same total combined path length, aligning their remaining distances.

Q1294. How do you remove duplicates from an unsorted linked list?

A: Use a hash set for $O(n)$ expected time, or nested traversal for $O(n^2)$ time and $O(1)$ extra space.

Q1295. How do you clone a linked list with random pointers?

A: Use a hash map from original nodes to cloned nodes, or interleave clones with originals for $O(1)$ auxiliary space.

Q1296. Why is linked-list cache locality usually poor?

A: Nodes may be scattered in memory, causing pointer chasing and cache misses.

Q1297. What is a monotonic stack?

A: A stack maintained in increasing or decreasing order to answer nearest-greater or nearest-smaller queries

efficiently.

Q1298. How do you find the next greater element?

A: Traverse from right to left, removing stack elements that cannot be the next greater candidate.

Q1299. How do you find the previous smaller element?

A: Traverse left to right and pop values that are not smaller than the current value.

Q1300. What is the largest rectangle in a histogram approach?

A: Use a monotonic increasing stack to determine the maximal width for each bar.

Q1301. Why does the histogram algorithm run in $O(n)$?

A: Each bar is pushed and popped at most once.

Q1302. How can a stack validate parentheses?

A: Push opening brackets and match each closing bracket against the top.

Q1303. How do you find duplicate brackets?

A: On a closing parenthesis, inspect whether the enclosed expression contains an operator before popping.

Q1304. What is a stack-based expression evaluator?

A: Use operand and operator stacks or a postfix representation to respect precedence and associativity.

Q1305. Why is recursion conceptually a stack?

A: Each active function call stores local state and return information on the call stack.

Q1306. What is stack overflow?

A: Failure caused by exhausting available call-stack memory, often due to excessive recursion.

Q1307. What is a deque?

A: A double-ended queue supporting insertion and removal at both the front and back.

Q1308. When is a deque preferable to a queue?

A: When both ends need efficient access, such as monotonic-window algorithms.

Q1309. What is a circular queue?

A: A queue whose storage wraps around from the end back to the beginning.

Q1310. Why use a circular queue?

A: It reuses freed positions and avoids shifting elements.

Q1311. How can a queue be implemented using two stacks?

A: Use one stack for incoming elements and another for outgoing elements; transfer only when the outgoing stack is empty.

Q1312. What is the amortized complexity of queue operations using two stacks?

A: $O(1)$ amortized per operation.

Q1313. How can a stack be implemented using queues?

A: Use one or two queues and rearrange elements so the required end behaves like a stack top.

Q1314. What is a priority queue?

A: An abstract data type that returns the highest- or lowest-priority element efficiently.

Q1315. What is the difference between a queue and a priority queue?

A: A normal queue follows insertion order; a priority queue follows priority.

Q1316. Why is BFS naturally implemented with a queue?

A: It processes vertices in layers, ensuring nodes at smaller distance are processed first in unweighted graphs.

Q1317. What is tree height?

A: The length of the longest downward path from a node to a leaf, with the exact convention depending on whether height counts edges or nodes.

Q1318. What is tree depth?

A: The distance from the root to a node under the chosen edge/node convention.

Q1319. What is the diameter of a tree?

A: The maximum distance between any two nodes.

Q1320. How can tree diameter be computed in $O(n)$?

A: Compute subtree heights with DFS and update the best sum of the two largest child heights at each node.

Q1321. What is tree serialization?

A: Encoding a tree into a sequence that preserves enough information to reconstruct its structure.

Q1322. Why are null markers needed in many tree serializations?

A: Without structural markers, different tree shapes can produce the same traversal sequence.

Q1323. What is a threaded binary tree?

A: A tree that uses otherwise-null child pointers to store traversal-related links.

Q1324. What is a complete binary tree?

A: Every level except possibly the last is full, and the last level is filled from left to right.

Q1325. What is a full binary tree?

A: Every node has either zero or two children.

Q1326. What is a perfect binary tree?

A: Every internal node has two children and all leaves are at the same depth.

Q1327. What is the successor of a node in a BST?

A: The smallest key strictly greater than the node's key.

Q1328. What is the predecessor of a node in a BST?

A: The largest key strictly smaller than the node's key.

Q1329. How do you find BST successor without parent pointers?

A: If the node has a right subtree, take its minimum; otherwise track the deepest ancestor for which the node lies in the left subtree.

Q1330. How do you validate a BST robustly?

A: Use lower and upper bounds while recursively enforcing that every node lies within its valid range.

Q1331. Why can comparing only a node with its children fail to validate a BST?

A: A node can satisfy local child relationships while violating an ancestor's global ordering constraint.

Q1332. How do you find kth smallest in a BST?

A: Perform iterative or recursive inorder traversal and stop after visiting k nodes.

Q1333. Why does inorder traversal produce sorted order in a BST?

A: All left-subtree keys precede the node, and all right-subtree keys follow it.

Q1334. What is a balanced BST?

A: A BST whose height is kept near logarithmic in the number of nodes.

Q1335. What is an AVL tree?

A: A self-balancing BST where the height difference of left and right subtrees is bounded at each node.

Q1336. What is a red-black tree?

A: A self-balancing BST using node colors and invariants to keep height $O(\log n)$.

Q1337. What is heapify?

A: The process of restoring the heap property after a local violation.

Q1338. What is bottom-up heap construction complexity?

A: $O(n)$, not $O(n \log n)$, because most nodes are near the leaves and require little work.

Q1339. How do you get the kth largest element using a min-heap?

A: Maintain a min-heap of size k ; discard the smallest whenever the heap exceeds k .

Q1340. How do you get the kth smallest using a max-heap?

A: Maintain a max-heap of size k and discard the largest when the size exceeds k .

Q1341. What is a d-ary heap?

A: A heap where each node has up to d children instead of two.

Q1342. How does increasing heap arity affect operations?

A: It can reduce height but increases the number of children examined during sift-down.

Q1343. Can a heap efficiently search for an arbitrary value?

A: No; heaps guarantee ordering only between parents and children, so arbitrary search is generally $O(n)$.

Q1344. What is heap sort's worst-case complexity?

A: $O(n \log n)$.

Q1345. Is heap sort stable?

A: Standard heap sort is not stable.

Q1346. Why is a heap useful in streaming problems?

A: It maintains a compact set of extreme candidates without storing or sorting the entire stream.

Q1347. What is a DAG?

A: A directed acyclic graph.

Q1348. Why does a DAG have a topological ordering?

A: Because every finite DAG contains at least one vertex with indegree zero, enabling repeated removal.

Q1349. How can topological sort detect a cycle?

A: If fewer than V vertices are processed, the graph contains a cycle.

Q1350. What is Kahn's algorithm?

A: A BFS-like topological sort that repeatedly removes zero-indegree vertices.

Q1351. What is DFS-based topological sorting?

A: Perform DFS and push each vertex after processing all outgoing neighbors; reverse the finishing order.

Q1352. What is strongly connected component?

A: A maximal set of vertices in a directed graph where every vertex can reach every other.

Q1353. What is Kosaraju's algorithm?

A: Run DFS to obtain finishing order, reverse all edges, then DFS in reverse finishing order to identify SCCs.

Q1354. What is Tarjan's SCC algorithm?

A: A single-DFS algorithm using discovery times, low-link values, and a stack to identify SCCs.

Q1355. What is a bridge in an undirected graph?

A: An edge whose removal increases the number of connected components.

Q1356. What is an articulation point?

A: A vertex whose removal increases the number of connected components.

Q1357. What is the difference between Dijkstra and BFS?

A: BFS finds shortest paths in unweighted graphs; Dijkstra handles nonnegative weighted edges.

Q1358. Can Dijkstra handle negative edges?

A: No; negative edges invalidate its greedy finalization assumption.

Q1359. What algorithm handles negative edges?

A: Bellman-Ford can handle negative edge weights and can detect reachable negative cycles.

Q1360. What is Floyd-Warshall?

A: A dynamic-programming algorithm for all-pairs shortest paths in $O(V^3)$.

Q1361. What is a negative cycle?

A: A cycle whose total edge weight is negative, allowing path cost to decrease indefinitely by repeated traversal.

Q1362. What is 0-1 BFS?

A: A shortest-path algorithm for graphs whose edge weights are only 0 or 1, using a deque.

Q1363. Why does 0-1 BFS use a deque?

A: Zero-weight edges are processed from the front while one-weight edges are added to the back.

Q1364. What is a shortest-path DAG?

A: A DAG representing edges consistent with shortest-distance progression, often enabling linear-time DP after topological ordering.

Q1365. What is multi-source BFS?

A: BFS initialized with multiple sources at distance zero to compute the nearest-source distance.

Q1366. What is a useful application of multi-source BFS?

A: Computing distance to the nearest marked cell, facility, infected node, or other source set.

Q1367. Why does Kruskal's algorithm use DSU?

A: DSU efficiently checks whether adding an edge would connect vertices already in the same component.

Q1368. Why does Kruskal sort edges?

A: It considers edges in nondecreasing weight order, selecting safe minimum-weight connections.

Q1369. What is Prim's algorithm's central idea?

A: Grow one connected tree by repeatedly selecting the cheapest edge crossing from the current tree to an outside vertex.

Q1370. Can an MST be unique?

A: Yes; distinct edge weights guarantee a unique MST.

Q1371. Can multiple MSTs exist?

A: Yes, especially when equal-weight edges provide alternative optimal connections.

Q1372. What is the cut property?

A: A minimum-weight edge crossing any cut is safe for some MST; with suitable uniqueness conditions it can be forced.

Q1373. What is the cycle property?

A: A strictly heaviest edge on a cycle cannot belong to any MST.

Q1374. What is DSU path compression?

A: It shortens parent paths during find operations so future queries become faster.

Q1375. What is union by rank/size?

A: Attach the smaller or shallower set beneath the larger one to limit tree height.

Q1376. What is DSU complexity?

A: With path compression and union by rank/size, operations are $O(\alpha(n))$ amortized, effectively constant for practical sizes.

Q1377. What is an exchange argument in greedy algorithms?

A: A proof technique showing that a greedy choice can replace a choice in an optimal solution without making the solution worse.

Q1378. What is the greedy-choice property?

A: There exists an optimal solution that begins with or includes a locally optimal choice.

Q1379. What is optimal substructure?

A: An optimal solution contains optimal solutions to relevant subproblems.

Q1380. Why is greedy not automatically valid when optimal substructure exists?

A: Optimal substructure alone does not prove that the local greedy choice is globally safe.

Q1381. Why does interval scheduling work greedily by earliest finish time?

A: Choosing the earliest finishing compatible interval leaves the maximum remaining room for future intervals.

Q1382. Why does fractional knapsack use value/weight ratio?

A: Because items can be divided, taking the highest ratio first maximizes value for each unit of capacity.

Q1383. Why does the same ratio strategy fail for 0/1 knapsack?

A: Items cannot be split, so locally best ratios can prevent the globally optimal combination.

Q1384. What is Huffman coding?

A: A greedy algorithm that repeatedly combines the two least frequent symbols to construct an optimal prefix code.

Q1385. What is a prefix-free code?

A: No valid codeword is a prefix of another codeword.

Q1386. Why does Huffman coding support unambiguous decoding?

A: Prefix-free codewords can be decoded without separators because no codeword is the prefix of another.

Q1387. What is memoization?

A: Top-down dynamic programming that caches results of already solved states.

Q1388. What is tabulation?

A: Bottom-up dynamic programming that fills states in an order where dependencies are already computed.

Q1389. When is memoization preferable?

A: When many theoretical states are unreachable and recursion naturally explores only needed states.

Q1390. When is tabulation preferable?

A: When iterative control, predictable memory use, or easier stack avoidance is desirable.

Q1391. What is state compression in DP?

A: Reducing the stored dimensions when a state depends only on a small portion of previous states.

Q1392. When can a DP dimension be reduced?

A: When the transition for the current state depends only on a limited previous layer or fixed range of earlier

states.

Q1393. What is a transition in DP?

A: The recurrence describing how one state is computed from other states.

Q1394. What is a base case in DP?

A: A state whose answer is known directly and starts the recurrence.

Q1395. What is a DP invariant?

A: A precise meaning assigned to every DP state that ensures the recurrence computes the intended quantity.

Q1396. What is digit DP?

A: A DP technique for counting or optimizing numbers satisfying digit-based constraints over a numeric range.

Q1397. What is the Euclidean algorithm?

A: A method for gcd based on $\text{gcd}(a,b) = \text{gcd}(b, a \bmod b)$.

Q1398. Why is Euclid's algorithm fast?

A: The remainder sequence decreases rapidly; its complexity is $O(\log \min(a,b))$.

Q1399. What is the extended Euclidean algorithm?

A: It computes $\text{gcd}(a,b)$ and coefficients x,y such that $ax + by = \text{gcd}(a,b)$.

Q1400. When does a modular inverse exist?

A: For a modulo m , an inverse exists exactly when $\text{gcd}(a,m)=1$.

Q1401. What is Euler's totient function?

A: $\varphi(n)$ counts integers from 1 through n that are coprime with n .

Q1402. What is modular exponentiation?

A: Computing $a^b \bmod m$ efficiently using repeated squaring.

Q1403. What is fast exponentiation complexity?

A: $O(\log b)$ multiplications for exponent b .

Q1404. What is a prime factorization?

A: Representing an integer as a product of prime powers.

Q1405. How can numbers up to n be factorized using a smallest-prime-factor array?

A: Precompute the smallest prime factor for each integer, then repeatedly divide by its stored factor.

Q1406. What is the Sieve of Eratosthenes complexity?

A: $O(n \log \log n)$ time with $O(n)$ auxiliary space.

Q1407. What is a trie?

A: A tree-like structure that stores strings character by character along shared prefixes.

Q1408. What is the time complexity of trie insertion?

A: $O(L)$, where L is the string length, assuming constant-time child access.

Q1409. What is the time complexity of trie search?

A: $O(L)$.

Q1410. How does a trie support prefix queries?

A: Traverse the prefix node, then inspect or enumerate its descendants.

Q1411. What is a compressed trie?

A: A trie that merges chains of single-child nodes to reduce structural overhead.

Q1412. What is a suffix trie?

A: A trie containing all suffixes of a string, useful conceptually for substring queries but potentially memory-heavy.

Q1413. What is a suffix array?

A: A sorted array of all suffix starting positions of a string.

Q1414. What is the LCP array?

A: An array storing longest common prefix lengths between adjacent suffixes in suffix-array order.

Q1415. What is a ternary search tree?

A: A string-search structure where each node has less-than, equal, and greater character branches.

Q1416. Why can tries consume much more memory than hash tables?

A: Each character transition may require node and pointer/array overhead, especially when the alphabet is sparse.

Q1417. What is KMP used for?

A: Finding occurrences of a pattern in a text in $O(n+m)$ time.

Q1418. What is the LPS array in KMP?

A: For each pattern prefix, it stores the length of the longest proper prefix that is also a suffix.

Q1419. Why does KMP avoid rechecking text characters?

A: The LPS information determines how far the pattern can shift after a mismatch without invalidating previous matches.

Q1420. What is the Z-array?

A: $Z[i]$ is the length of the longest substring starting at i that matches the prefix of the string.

Q1421. How can Z-algorithm perform pattern matching?

A: Build Z over pattern + separator + text and identify positions whose Z value equals the pattern length.

Q1422. What is Rabin-Karp?

A: A rolling-hash based pattern-matching algorithm that compares hash values before verifying candidate matches.

Q1423. What is the expected complexity of Rabin-Karp?

A: Typically $O(n+m)$ expected time with a good hash, while worst-case behavior can degrade due to collisions.

Q1424. What is a rolling hash?

A: A hash that can be updated efficiently when a fixed-size window moves.

Q1425. Why can hashing alone not prove string equality?

A: Different strings can collide, so candidate hash matches may require direct verification.

Q1426. What is lexicographical order?

A: Dictionary-like ordering determined by the first differing character, with prefix rules when one string ends earlier.

Q1427. What is a segment tree?

A: A tree over array intervals supporting range queries and point or range updates efficiently.

Q1428. What is the typical segment-tree query complexity?

A: $O(\log n)$ for many associative range operations.

Q1429. What is the typical segment-tree point-update complexity?

A: $O(\log n)$.

Q1430. What is lazy propagation?

A: Deferring range updates by storing pending operations at internal nodes until their values are needed.

Q1431. Why is lazy propagation useful?

A: It reduces range-update complexity from potentially $O(n)$ to $O(\log n)$ for suitable operations.

Q1432. What is a Fenwick tree?

A: A compact structure for prefix aggregates, especially sums, with $O(\log n)$ updates and queries.

Q1433. Why is Fenwick tree memory efficient?

A: It stores one value per array position rather than an explicit tree of nodes.

Q1434. Can a Fenwick tree directly support arbitrary range minimum updates?

A: Not with the standard sum-oriented structure; the operation must satisfy the required update/query properties.

Q1435. What is a sparse table?

A: A static range-query structure that precomputes answers for power-of-two intervals.

Q1436. When is a sparse table preferable?

A: For static arrays with many range queries and no updates, especially idempotent operations such as minimum or maximum.

Q1437. What is divide and conquer?

A: Break a problem into smaller independent subproblems, solve them recursively, and combine their results.

Q1438. What is the master theorem used for?

A: Analyzing many divide-and-conquer recurrences of the form $T(n) = aT(n/b) + f(n)$.

Q1439. Why does merge sort need $O(n)$ auxiliary memory in its common array implementation?

A: The merge step typically needs temporary storage to combine two sorted ranges.

Q1440. Why can divide-and-conquer recursion cause stack usage?

A: Each recursive level stores function state until its subproblems return.

Q1441. What is tail recursion?

A: A recursive call is the final operation of the function, allowing some languages/compiler to optimize it, though C++ does not guarantee tail-call elimination.

Q1442. What is meet-in-the-middle?

A: Split a search space into two halves, enumerate each half, then combine results efficiently.

Q1443. When is meet-in-the-middle useful?

A: For exponential problems where n is too large for 2^n but small enough for roughly $2^{(n/2)}$ enumeration.

Q1444. What is randomized algorithm?

A: An algorithm that uses random choices during execution, potentially improving expected performance or avoiding adversarial cases.

Q1445. What is Monte Carlo algorithm?

A: A randomized algorithm with bounded or predictable runtime that may have a small probability of an incorrect result.

Q1446. What is Las Vegas algorithm?

A: A randomized algorithm that always returns a correct result but has random running time.

Q1447. What is a cache-friendly algorithm?

A: An algorithm whose memory-access pattern makes effective use of CPU caches and locality.

Q1448. Why can two $O(n)$ algorithms have different practical performance?

A: Constant factors, cache locality, branch prediction, allocation overhead, and memory layout can differ substantially.

Q1449. What is spatial locality?

A: Accessing memory addresses close to recently accessed addresses.

Q1450. What is temporal locality?

A: Reusing the same memory locations within a short period.

Q1451. Why are arrays often faster than linked lists?

A: Arrays provide contiguous storage, good cache locality, and direct indexing.

Q1452. What is false sharing?

A: In multithreaded systems, independent variables on the same cache line cause unnecessary cache-coherence traffic.

Q1453. What is branch prediction?

A: CPU hardware predicts the direction of conditional branches to keep instruction pipelines busy.

Q1454. Why can branchless code sometimes help?

A: It may reduce costly branch mispredictions, though it is not universally faster.

Q1455. What is memory bandwidth?

A: The rate at which data can be transferred between memory and processors or other components.

Q1456. Why can an algorithm be memory-bound?

A: Its performance may be limited more by data movement than by arithmetic or logical computation.

Q1457. What is the difference between an algorithm and a data structure?

A: An algorithm is a procedure for solving a problem; a data structure organizes data to support operations efficiently.

Q1458. Why should data-structure choice follow operation requirements?

A: Different structures optimize different operations, so the workload determines the appropriate trade-offs.

Q1459. What is a trade-off between time and space?

A: Extra memory can reduce computation time, while lower memory usage may require more computation.

Q1460. What is preprocessing?

A: Work performed before answering queries or processing the main input to make later operations faster.

Q1461. What is online processing?

A: Processing data incrementally as it arrives without requiring the complete input beforehand.

Q1462. What is offline processing?

A: Using knowledge of the complete input or query set to reorder or preprocess operations for efficiency.

Q1463. What is a streaming algorithm?

A: An algorithm designed to process data sequentially with limited memory, often without storing the entire dataset.

Q1464. What is an invariant-based solution?

A: A solution whose correctness is maintained by a property that remains true throughout execution.

Q1465. What is the best way to explain an algorithm in an interview?

A: State the idea, invariant or reasoning, steps, complexity, edge cases, and then implement it clearly.

Q1466. What should you do when you cannot derive an optimal solution immediately?

A: Clarify constraints, propose a correct brute-force approach, identify the bottleneck, and iteratively optimize it.

Q1467. What is the difference between connected and strongly connected?

A: Connected usually refers to undirected reachability; strongly connected in directed graphs requires mutual reachability.

Q1468. What is a bipartite graph?

A: A graph whose vertices can be divided into two sets such that every edge connects vertices from different sets.

Q1469. How can bipartiteness be tested?

A: Use BFS or DFS coloring with two colors and detect an edge connecting same-colored vertices.

Q1470. What does an odd cycle imply?

A: A graph containing an odd cycle is not bipartite.

Q1471. How do you find connected components?

A: Run DFS or BFS from every unvisited vertex and count traversals.

Q1472. What is graph density?

A: A measure of how many edges exist relative to the maximum possible number of edges.

Q1473. When is an adjacency matrix useful?

A: For dense graphs or when $O(1)$ edge-existence checks are important.

Q1474. When is an adjacency list preferable?

A: For sparse graphs, because it uses $O(V+E)$ space.

Q1475. What is edge list representation?

A: A collection of edges, typically storing endpoint pairs and optionally weights.

Q1476. Why can adjacency matrices be expensive?

A: They require $O(V^2)$ memory regardless of the actual number of edges.

Q1477. What is rolling median?

A: The median maintained over a moving window or streaming sequence, commonly using two heaps.

Q1478. How do two heaps maintain a median?

A: Keep a max-heap for the lower half and a min-heap for the upper half, balancing their sizes.

Q1479. What is the invariant for two-heap median?

A: All values in the lower heap are $\leq$ all values in the upper heap, and their sizes differ by at most one.

Q1480. How do you find the top k frequent elements?

A: Count frequencies with a hash map and use a heap or bucket-based approach.

Q1481. What is bucket-based top-k frequency?

A: Group values by frequency and scan buckets from high to low.

Q1482. How do you merge k sorted arrays efficiently?

A: Use a min-heap containing one current element from each array.

Q1483. What is external sorting?

A: Sorting data too large for memory by sorting chunks and merging them from external storage.

Q1484. Why is external merge sort appropriate for huge datasets?

A: It minimizes random I/O and performs sequential reads/writes during merging.

Q1485. What is a k-way merge?

A: Merging k sorted streams by repeatedly selecting the smallest current element, usually with a min-heap.

Q1486. What is reservoir sampling?

A: A streaming technique that selects a uniformly random sample of fixed size from an unknown-length stream.

Q1487. What is a sentinel node in a linked list?

A: A dummy node placed at a boundary to simplify insertion and deletion logic.

Q1488. Why use a dummy node?

A: It removes special cases involving operations at the head of the list.

Q1489. What is pointer aliasing?

A: Multiple pointers or references refer to the same memory location or object.

Q1490. Why is pointer aliasing important in algorithm code?

A: Mutating through one alias can unexpectedly change what another alias observes.

Q1491. What is a dangling pointer?

A: A pointer referring to an object whose lifetime has ended.

Q1492. What is a memory leak?

A: Allocated memory that remains unreachable and therefore cannot be reclaimed by normal ownership logic.

Q1493. What is double free?

A: Attempting to release the same dynamically allocated resource more than once.

Q1494. Why are ownership rules important in linked structures?

A: Incorrect ownership can cause leaks, dangling pointers, or double deletion.

Q1495. What is recursion depth risk in tree algorithms?

A: A highly skewed tree can make recursive depth $O(n)$, potentially exhausting the call stack.

Q1496. How can deep recursive DFS be avoided?

A: Use an explicit stack to perform iterative DFS.

Q1497. What is a randomized test for an algorithm?

A: Generate many inputs, compare the optimized algorithm against a trusted brute-force implementation, and detect mismatches.

Q1498. What is differential testing?

A: Compare two independent implementations on the same inputs to uncover correctness bugs.

Q1499. Why are edge cases important in DSA interviews?

A: Many failures occur at empty input, one element, duplicates, extreme values, boundaries, or overflow cases.

Q1500. What integer issue commonly breaks prefix sums?

A: Accumulated values may exceed 32-bit range, so a wider integer type may be required.

Q1501. Why should multiplication overflow be considered?

A: Even if final answers fit a type, intermediate products may overflow before reduction or comparison.

Q1502. What is integer division truncation?

A: For integer operands, the fractional part is discarded according to the language's integer-division rules.

Q1503. Why should $\text{mid} = (l+r)/2$ be avoided in some contexts?

A: $l+r$ can overflow; use $l + (r-l)/2$.

Q1504. What is a comparator pitfall in sorting?

A: A comparator that violates strict weak ordering can produce undefined or incorrect behavior.

Q1505. What is strict weak ordering?

A: A comparator relation satisfying properties required by standard sorting algorithms, including irreflexivity and transitivity constraints.

Q1506. Why should constraints drive algorithm selection?

A: Input size and operation limits determine whether $O(n^2)$, $O(n \log n)$, $O(n)$, or more specialized approaches are feasible.